THE BOWSERBOY DS BOOK

Two Screens, One Little Escape

Development journey
and engineering



First NDS edition / November 2026
From local cartridge to a living, two-screen desk.

Contents

A separate NDS book. Source-pinned chapters, worked examples and explained exercises.

Before opening the DS 3

Part 1 - Becoming a console 5

ND01 - A second console 6
ND02 - The development journey 12

Part 2 - Making it run 18

ND03 - Static host, native core 19
ND04 - Cartridge, loading and power 24
ND05 - One framebuffer, two live screens 29
ND06 - Touch, controls and the stylus 34
ND07 - Saves, identity and recovery 39

Part 3 - Making it feel physical 45

ND08 - The DS on the desk 46
ND09 - Algorithms and data structures 53
ND10 - Design principles and failure boundaries 58

Part 4 - Evidence and maintenance 63

ND11 - Tests, releases and honest evidence 64
ND12 - Visual documentation 70
ND13 - The maintainer field guide 77
Source register and limits 83

Before opening the DS

This is the separate BowserBOY DS book: a development journey followed by an engineering explanation of the Nintendo DS experience. It does not import the original console's hardware, emulator or historical teaching chapters. The existing book, *Building a Little Escape*, remains an independent, preserved publication.

The central problem is joining an existing native emulator to an expressive, recoverable physical-feeling browser appliance. Two screens, a hinged body, precise touch, a clipped stylus and a local cartridge are not separate decorations. Their coordinate systems, clocks and failure states must agree.

What this edition means

The implementation cutoff is `61845372142b41992916f3732ea102cc0f4c8edf`, November 7, 2026. The new manuscript describes that source; its own publication can occur later. Historical workstreams retain their original states and failures. A later supported-console designation does not retroactively turn every earlier prototype into a fully qualified release.

DS direct boot uses EmulatorJS 4.2.3 and DeSmuME. Original Nintendo DS cartridges are in scope; DSi-enhanced/DSi-only files are rejected. No commercial ROM, BIOS or firmware is included. Local cartridge selection does not upload the ROM to a gameplay server, but first runtime startup needs CDN access. No universal game-compatibility, multiplayer, physical-phone performance or permanent browser-storage guarantee is made.

The cutoff's hosted suite declared 119 cases: 98 passed and 21 were explicitly parked/skipped. Zero failures in that run does not mean 119 passes or repaired coverage. Some original workstreams remain open for broader qualification or owner review. Chapter 11 explains how those facts coexist with a supported, published console.

Choose a route

Read chapters 01-02, then 11-12, for the causal journey and its evidence. Read chapters 03-07 to follow a cartridge through the runtime, display, input and save boundaries. Chapters 08-10 explain the physical scene, algorithms and design choices. Chapter 13 is a practical change-and-recovery field guide.

Diagrams explain ownership or ordering; captions state what their arrows mean. Worked examples and changed-condition retries teach the mechanism. Questions have explained answers, not implied records of a reader's progress. The source register binds claims to public repository files and symbols. This is not an exhaustive source atlas or a Nintendo DS CPU textbook.

Pictures and evidence

The logo is existing supplied DS artwork, not newly generated cover art. Scene figures use existing public showcase GIFs; print embeds only frame zero. The HTML reading edition uses the same labeled stills. The animated originals remain in the repository. Their game displays replay recorded native opening footage during real scene interactions; these are not live game-input tests. No raw recording or private capture path is part of this book.

Code inspection, automated checks, visual inspection, actual public delivery and human approval are different evidence layers. Rendering this edition does not rerun historical gameplay checks or establish aesthetic approval.

PART 1

Becoming a console

1. A second console
2. The development journey

PART 1 / BECOMING A CONSOLE

ND01

A second console

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND01 / A SECOND CONSOLE

Contents

A second console

What makes a second console more than another model?

The motivating question is not whether a DS-shaped object can sit on the desk. It is whether selecting it establishes a coherent cartridge, display, input and persistence boundary without disturbing the original console. The outcome of this chapter is a vocabulary for that boundary: what the implementation supports, what its first prototype established, and what neither a successful demo nor a product label can establish.

This account stops at implementation `61845372142b41992916f3732ea102cc0f4c8edf`. It describes recorded evidence; authoring this chapter did not execute new emulator tests.

The prototype answered a deliberately smaller question

Workstream 79 requested playable Nintendo DS emulation beside the retained console, while keeping static hosting and separate runtime lifecycles. Its original technical result used pinned EmulatorJS 4.2.3/DeSmuME to boot the project's original Twin Lab homebrew without supplied BIOS or firmware. Real native frames reached both modeled LCDs; buttons and lower-screen stylus input reached the core. That was substantially more than a decorative screen animation. It was still a local prototype result, not a commercial compatibility matrix, physical-phone qualification or publication. [S1]

The distinction matters because the model and the runtime advanced at different rates. The first silhouette, disconnected hinge and loose pen-like stylus were rejected while verified emulator and save behavior were preserved. Remodeling the shell did not require throwing away the native integration. Conversely, native output did not make the first model visually acceptable. The historical record explicitly separates those outcomes rather than retroactively making the initial attempt successful in every dimension. [S1]

That separation is an engineering asset. A failure in geometry should lead to geometry evidence; a failure in native input should lead to native evidence. Treating every visible problem as a core defect, or every passing core check as approval of the object, would erase the useful boundary.

Supported now does not mean universally compatible

Workstream 114 later changed the current product status: BowserBOY DS is a supported console, not a feature currently labeled “local prototype.” The change reconciled UI labels, runtime errors and documentation, and its public-copy verification was recorded through the monitored release. It did not add emulator functionality or certify additional games. Historical captions, filenames and work logs deliberately retained their provenance. [S2]

Thus both statements can be true: the first delivery was a local prototype; the current product presents DS as supported. The old workstream's open device/aesthetic qualifications are not evidence that current UI should still call it a prototype. Equally, removing that label cannot turn a desktop viewport check into a physical-device performance result.

The current acceptance boundary is concrete. Authorized, uncompressed `.nds` files must have an original-DS header, valid ARM program ranges and a size between 512 bytes and 512 MiB. DSi-enhanced and DSi-only inputs are rejected. The runtime options select stacked screens, no gap, 256×192 internal LCD resolution and one core, with external BIOS and BIOS-menu boot disabled. These are configuration and validation facts, not a promise that every structurally valid cartridge boots. [S3, S4]

Direct boot trades a simpler static-hosted launch for a compatibility boundary. Twin Lab demonstrates one BIOS-free path. It does not show that every game's startup assumptions are satisfied. The guide says this explicitly and distinguishes supplied user cartridges from automated fixtures. A maintainer should preserve that wording even after a particular user-supplied opening has appeared in public media. [S4]

Follow ownership through the selection boundary

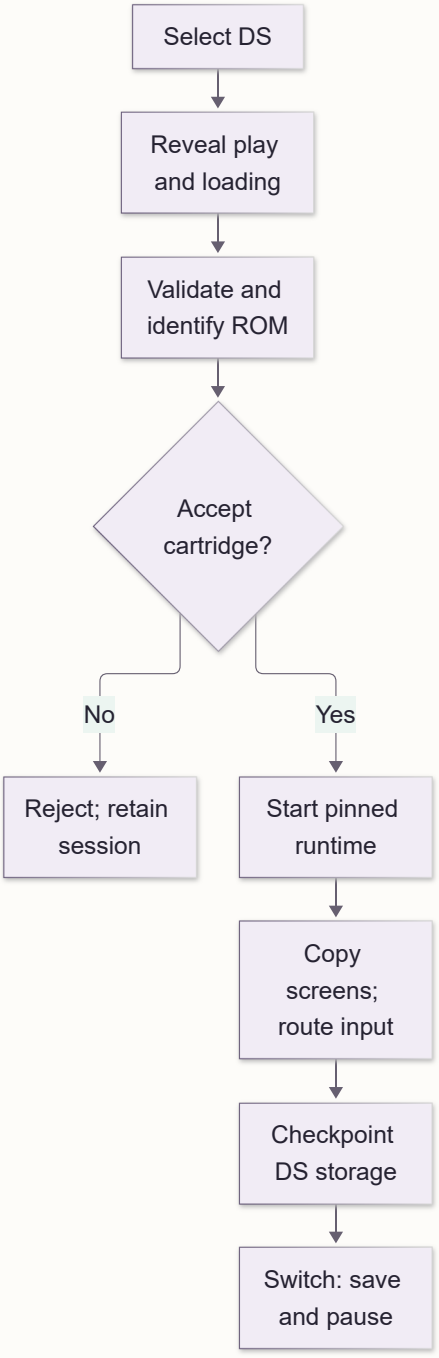


Figure ND01-01. The console boundary follows cartridge identity and lifecycle, not merely the selected model's color.

The diagram combines the immediate-loading contract with the established lifecycle contract. DS play appears before full-file reading and hashing, so a slow identification step no longer leaves the cartridge sidebar as the only apparent response. The lower LCD supplies loading feedback;

reduced motion keeps that graphic static. None of this accelerates the runtime download. It makes pending work legible and preserves cancellation and rejection behavior. [S5]

Persistence is equally part of the console identity. The DS has its own database and checksummed `.bbds-save` backups. Switching checkpoints and pauses the inactive runtime; simultaneous emulation was not required by the original acceptance contract. Power-on cold boot and same-ROM reload recovery are intentionally different operations. Without that distinction, “resume” could accidentally become a promise that a checkpoint survives replacement by a newer live-session save. Workstream 79 records an actual correction to that mistaken testing expectation. [S1]

ND01-W01 - Read a successful boot without overclaiming

Suppose the original Twin Lab demo boots and a mark appears on the lower physical LCD after touch input. A useful receipt identifies the fixture, runtime version, native output and displayed output. It supports the claim that this input-to-display path worked in that environment.

Now consider the next inference: “Any accepted `.nds` file will work.” The inference fails because header validation establishes structure, not the behavior of all programs accepted by that structure. The implementation hashes the full file to identify it only after header checks; it does not execute a compatibility proof. [S3]

Similarly, a 390-pixel desktop-browser layout result supports responsive framing, not physical-phone latency or thermal behavior. A later successful Pages gate establishes the retained hosted scenarios at its exact revision. It cannot retroactively qualify every prototype scenario, particularly the 21 cases still explicitly parked at this book's cutoff. [S1, S6]

ND01-R01 - Retry the boundary, not the whole product

Workstream 115 provides a practical retry model. A rejected replacement must report its error without losing an existing DS session; a cancelled native startup must stop loading and leave a usable retry route. The recorded focused cases exercised held upload reading, native handoff, invalid replacement retention, reduced motion and cancellation. [S5]

The lesson is not “try again until green.” First classify where progress stopped: file identification, core startup, display handoff or session replacement. Preserve the last valid ownership state while retrying the failed operation. A loading animation is useful precisely because it does not pretend those states are already successful.

ND01-Q01 - Exercise: which statement belongs in a release note?

Explained answer

A local demo passes, a hosted gate passes its retained cases, and the UI now says BowserBOY DS. Choose between these summaries:

1. “DS is supported; direct-boot and parked-coverage limits remain.”

2. “DS compatibility and physical-phone performance are fully verified.”

Explain which additional evidence the second statement would require.

ND01-A01 - Explained answer

Back to question

The first statement respects separate product, compatibility and evidence boundaries. The second would require title-specific compatibility evidence and actual device qualification; neither follows from a label or a desktop viewport. It would also have to account honestly for parked scenarios. At cutoff the release has 119 declared cases, 98 PASS and 21 SKIP, not 119 demonstrated behaviors. Chapter 11 explains how those numbers and public deployment receipts fit together.

Source references

- **S1:** Prototype contract, local evidence and actual corrections, TODO 79, lines 11–86 and 111–126; correction log.
- **S2:** Current supported-console status and historical-copy boundary, TODO 114, lines 15–34.
- **S3:** Runtime options, structural validation and ROM digest, lines 4–45.
- **S4:** Current player-facing compatibility and direct-boot limits, lines 177–193.
- **S5:** Immediate loading, replacement retention and focused evidence, TODO 115, lines 16–46.
- **S6:** Explicit restoration contract and final skip partition, TODO 105, lines 30–36 and 101–108; 21-case partition.

PART 1 / BECOMING A CONSOLE

ND02

The development journey

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND02 / THE DEVELOPMENT JOURNEY

Contents

The development journey

Which failed hypothesis should change the design?

A DS on a physical-looking desk has several clocks and coordinate systems: pointer events, native input polling, framebuffer copying, scene rendering and GPU completion. The motivating question is how to determine which one failed without compensating in the wrong layer. This chapter's outcome is a causal reading of the journey, not a catalog of commits.

The account is pinned to `61845372142b41992916f3732ea102cc0f4c8edf`. Historical measurements below are receipts from the public record, not newly executed benchmarks.

From a second object to an articulated console

Workstream 79 established dual-screen output, native input and separate persistence, then reopened the model after rejection of its geometry, hinge and loose stylus. Its correction log also records pickup intersection, frozen inspection paints, scaled framebuffer splitting and lower-LCD direct-color behavior. These were different failures with different owners; a single “DS implemented” milestone would hide their causes. [S1]

Subsequent numbered workstreams refined the object into an interaction system. Independent console movement made placement a constraint rather than a fixed coordinate. Articulated inspection and locked gameplay orientation meant the same touchscreen had to remain accurate through body rotation, hinge motion, zoom and resizing. A picked-up pen then had to remain visually useful without blocking its own input ray. These dependencies explain why touch correctness could not be reduced to a single screenshot in the initial flat pose. [S2, S3]

Not every numbered idea became a feature. The boxed-cartridge library remained explicitly deferred and required title-specific redistribution permission. A physical cartridge prop therefore must not be narrated as a downloadable game catalog. The book selects these causal boundaries rather than exporting every intermediate tracking entry. [S4]

A pointer offset was not permission to move every touch

Workstream 88 began with a click near the lower-right LCD appearing substantially higher in Twin Lab. The tempting repair would have shifted host coordinates. Instead, the recorded investigation traced the native bridge and original homebrew, corrected the demo's Y conversion and restricted brush bounds, and retained physical UV mapping. Native corner tests separately exposed synthetic autofocus/blur behavior and inclusive viewport endpoint issues. [S5]

That distinction protects every other cartridge from a compensation for one fixture's bug. Physical antialiasing introduced another false signal: a corner between drawing-buffer pixels could not reasonably be required to contain one completely yellow pixel. The corrected probe used a small sampling footprint while retaining exact native brush assertions. This was fixture accuracy, not permission to accept arbitrary drawing offsets.

Quick taps then falsified a different assumption. Eight ordinary clicks lost marks even though coordinates were correct; down/up arrived only 0–0.3 ms apart. Earlier held-contact precision tests had not exercised that path. Two browser animation frames still lost touches. The pinned native frame counter demonstrated why: browser paint boundaries are not native input-poll boundaries. The repair retained contact edges across actual native frame changes, coalesced pending moves and discarded queued contacts on cancellation. [S5]

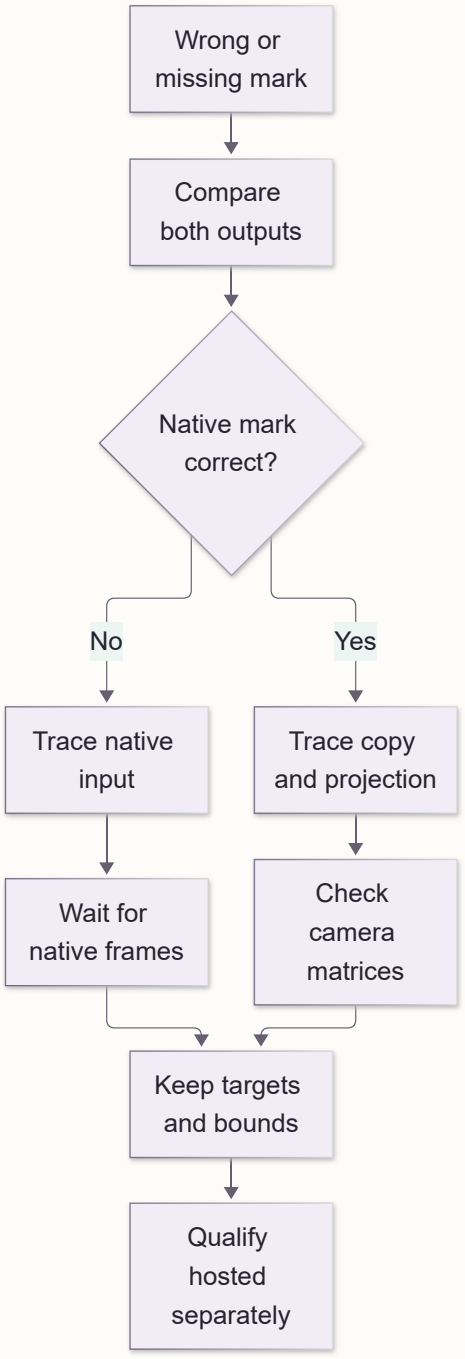


Figure ND02-01. A correct coordinate, a registered native contact and a timely displayed mark are three different observations.

Correct pixels can still arrive too late

The touch story did not end when native marks became correct. The public record contains software-rendered runs where native and copied pixels were right only after the unchanged 1500 ms quick-tap deadline. It also records a cached-controller experiment that failed and was rejected. A pass on hardware did not invalidate that software result. [S6]

Resize exposed another layer: camera positions changed while inverse and world matrices waited for an eligible room paint. A deferred-paint phone regression mapped `(32,80)` to `(65,56)` before correction. Synchronizing camera and active-model matrices at the change fixed projection without waiting for GPU rendering. A later held-pen regression showed that projected controls also needed synchronization after resize. [S6]

There was no evidence for a universal “GPU problem” that excused every failure. The record distinguishes delayed evaluation, native contact loss, stale projection and copying gated by room paints. An unqualified tiled/snapshot overlay experiment was removed because it still failed and exposed transition state before its full frame completed. Minimal full-frame fences and independent native-half copying were retained. The engineering trade-off was to decouple necessary work without publishing an incompletely rendered state. [S6]

ND02-W01 - Light the shell by moving the shell

Workstream 116 is a compact example of testing a physical explanation. The upright pose at `(5,5.1,8)` placed the front shell ahead of the lamp's emitting region. Lamp-only blue samples were only 9 on the lid and 10 on the deck. Enlarging tabletop bounce alone was ineffective.

Moving toward the light was promising but not sufficient: the first target `(-2,5.1,-2)` failed full-rotation clearance with a recorded gap of `-1.047`. The bounded correction moved the shared target to `(-2,5.1,-8)`. Final physical-shell samples reached blue 108–113 on the lid and 117–157 on the deck; all eight sampled rays hit shell and went black with every lamp off. No material, light or exposure edit was needed. [S7]

This result answers the original readability complaint without inventing a camera light or brightening native LCDs. Earlier paint work had already shown why hex-only trials were insufficient: rendered shell color depended on reflection and illumination, and measurements had to exclude green LCD pixels. Readability was a spatial problem here, not another request to repaint the console. [S8]

ND02-R01 - Retry a fixture without rewriting the observation

The new lighting fixture initially treated powered-off reflective glass as an unlit LCD. Its bounded repair used actual static status LCDs; native screen/input behavior remained separately qualified. Similarly, workstream 115 found full-file digest and storage lookup occurring before sidebar closure. Moving visible play/loading feedback earlier addressed perceived inactivity without claiming faster emulation. [S7, S9]

A useful retry changes the falsified assumption and preserves the question's original strength. It does not enlarge every deadline, lower renderer sharpness or reclassify a late result as timely. Later CI parking therefore remains unresolved evidence debt even where focused local checks succeeded.

ND02-Q01 - Exercise: repair a misleading quick-tap test

Explained answer

A test holds contact until a yellow native mark appears, then reports that ordinary taps work. Real short clicks still fail. What should the next regression measure, and why is adding a fixed host-screen offset wrong?

ND02-A01 - Explained answer

Back to question

Generate genuine short down/up contacts, inspect native and copied pixels separately, and relate edge retention to actual native frame changes. Keep cancellation and the original latency bound. A fixed coordinate offset cannot repair lost contact edges; it would corrupt correct UV mapping while leaving the timing failure intact.

The journey's durable result is this discipline, not an assertion that every scenario now passes. Workstream 114 establishes current supported console status without rewriting prototype history. Workstream 105 still preserves 21 skipped hosted cases, 18 CI-only and three also local. The next chapter on releases explains why both facts belong in the same engineering account. [S10]

Source references

- **S1:** Prototype corrections and remodel rejection, TODO 79, lines 111–126.
- **S2:** Movement purpose, TODO 85, lines 1–6; articulation purpose, TODO 87, lines 1–6.
- **S3:** Held-pen constraints, native evidence and Escape correction, TODO 89, lines 18–43.
- **S4:** Deferred library and redistribution boundary, TODO 80, lines 1–6.
- **S5:** Actual contact failure, native-frame repair and demo-coordinate correction, TODO 88, lines 174–225.
- **S6:** Rejected cached-controller trial, lines 55–69; projection and incomplete-frame experiments, lines 106–158.
- **S7:** Measured geometry, failed target and fixture correction, TODO 116, lines 25–56.
- **S8:** Rejected color/reflection trials and shell-only measurement, TODO 86, lines 17–43.
- **S9:** Immediate-loading diagnosis and verification, TODO 115, lines 25–46.
- **S10:** Supported status without new compatibility, TODO 114, lines 15–34; final skip partition, TODO 105, lines 101–108.

PART 2

Making it run

1. Static host, native core
2. Cartridge, loading and power
3. One framebuffer, two live screens
4. Touch, controls and the stylus
5. Saves, identity and recovery

PART 2 / MAKING IT RUN

ND03

Static host, native core

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND03 / STATIC HOST, NATIVE CORE

Contents

Static host, native core

Question: how does a static website become a DS without a game server? **Outcome:** trace a cartridge from a local browser file to a native emulator, and distinguish an application boundary from a security sandbox.

The host does not emulate the processors

BowserBOY DS combines a browser application, a Three.js physical presentation and a pinned EmulatorJS 4.2.3 adapter using DeSmuME. It is not a new ARM9 or ARM7 emulator. The website supplies the appliance around that existing core: cartridge selection, power, checked storage, input routing, scene geometry, status and recoverable errors.

Static hosting serves HTML, JavaScript and assets. The browser executes the application and emulator locally. The selected cartridge becomes a browser `File`, not an upload to a game-processing server. That distinction does not make first launch offline: `src/player.js` fetches the pinned runtime loader and its dependencies from the EmulatorJS CDN. CDN access is a real dependency. There is no service-worker-backed, guaranteed offline installation contract.

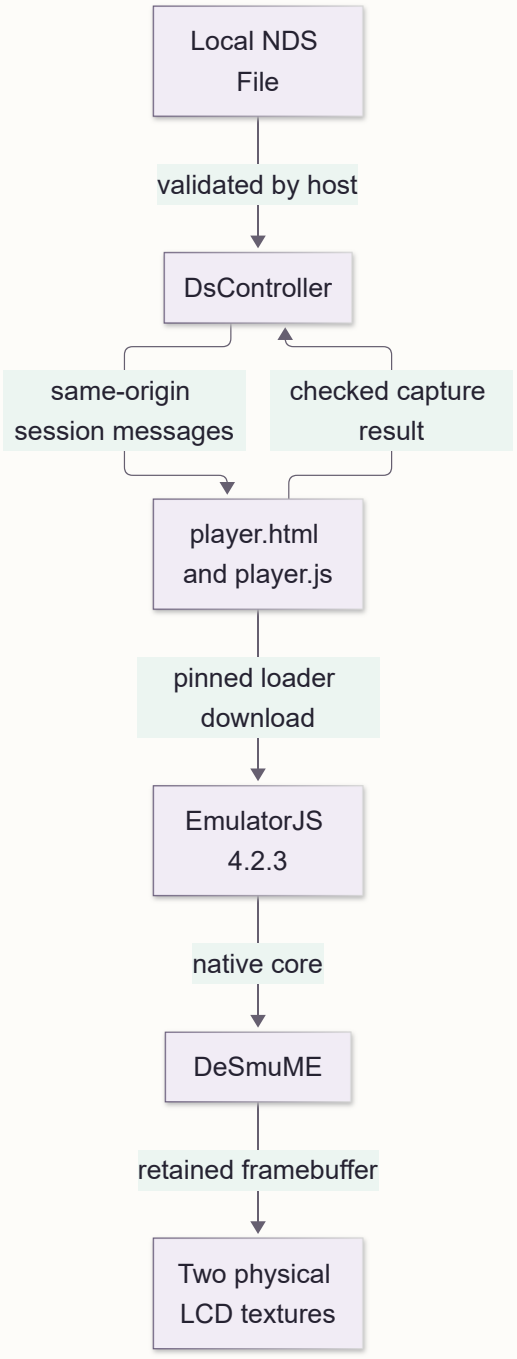


Figure ND03-01. Execution ownership. The CDN supplies runtime code; it is not the owner of local checkpoints or a remote gameplay service.

Three boundaries, three different promises

`DsController` owns the physical DS session. It knows the selected cartridge, checkpoint store, iframe, visible controls, screen materials and current power/pause state. It does not spread upstream `EJS_emulator` internals through the scene.

`player.js` is the version-sensitive adapter. It translates host commands into the pinned runtime's APIs, configures native options, retains the WebGL drawing buffer and translates native results back into messages. An upstream change therefore has a bounded inspection surface, although a version pin alone does not prove continued CDN availability.

The same-origin iframe separates document layout and runtime lifecycle from the desk. It is deliberately readable by the host, which obtains the native canvas through `contentDocument`. This is not a hostile-code security boundary. Same-origin code and downloaded runtime dependencies remain trusted code. Calling it a sandbox would overstate what the implementation provides.

Session identity is not optional decoration

The host creates a new UUID at launch. A host message handler accepts an event only when its source is the current iframe window, its origin matches the page and its session matches the current session UUID. The adapter likewise requires the parent window, matching origin and valid session. Initialization also checks that the ROM is a `Blob` and the game identity is a 64-digit lowercase hexadecimal digest.

These checks solve a practical asynchronous problem: an old runtime must not finish a command against a replacement cartridge. They do not authenticate arbitrary same-origin scripts or provide a capability boundary against all compromised page code.

Requests have separate UUIDs. `pending`, a `Map`, associates each command with its resolve/reject callbacks and ten-second timer. A result settles only its matching entry. `clear()` rejects pending work and removes the iframe; it does not leave callers waiting for results from a destroyed runtime.

ND03-W01 - A capture round trip

The Save action calls `command('capture')`. The controller registers the pending request before posting the message. The adapter validates the session, calls `getState()` and `getSaveFile()`, validates the resulting DS capture and sends a `bba:result` with the request ID.

The controller clears the timer and resolves the matching promise. Only then does the host attempt the IndexedDB transaction. Native capture and durable storage are separate steps: a successful capture is not a successful save. The save chapter follows that distinction through failure.

Failure is a message, not a blank screen

Loader download failure, invalid native video dimensions and runtime errors are surfaced as explicit failure messages. Launch has a 90-second deadline and a visible Cancel action. Post-start failure clears the runtime, powers the controller down and opens the cartridge tools with an error.

The adapter disables its own databases/local storage and hides unmanaged save/import controls. This avoids two competing persistence owners. It is not a blanket claim that every possible upstream feature is audited.

ND03-R01 - Replace the session before a result returns

Suppose request A belongs to a removed iframe, and cartridge B is now active. Even a well-formed delayed response from A cannot pass the current-window and session checks. The old promise is already rejected by `clear()`. A request ID without source/session checking would not provide that protection.

ND03-Q01 - Why not call DeSmuME directly from the scene?

Explained answer

ND03-A01 - Contain the unstable interface

Back to question

Direct calls would make rendering and physical interaction depend on version-specific emulator internals and its document lifetime. The adapter centralizes that coupling; messages give operations explicit completion and error paths. This improves maintainability, not automatic compatibility with future EmulatorJS versions.

Source trail

DsController: launch, command, message and clear. Adapter: initialize and message dispatch. Read the source register for the pinned edition boundary.

PART 2 / MAKING IT RUN

ND04

Cartridge, loading and power

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Cartridge, loading and power

Question: what should happen immediately after selecting a large ROM? **Outcome:** distinguish scene handoff, validation, insertion, native launch, checkpoint resume, pause and power cycling.

Give the player the console before doing expensive work

The important loading improvement is ordering, not a fictional speedup. `load(file)` closes the DS tools, selects DS, requests playing view and paints `READING LOCAL ROM` before awaiting cartridge identification. Large-file reading, hashing and native startup still take time, but the player is no longer stranded in the selection sidebar without visible progress.

This is asynchronous work on the page, not a claim that the ROM hashing was moved to a worker. Reading bytes and computing a digest can consume memory and browser resources. The observable contract is early handoff and a loading display while awaited work proceeds; it is not instant emulation.

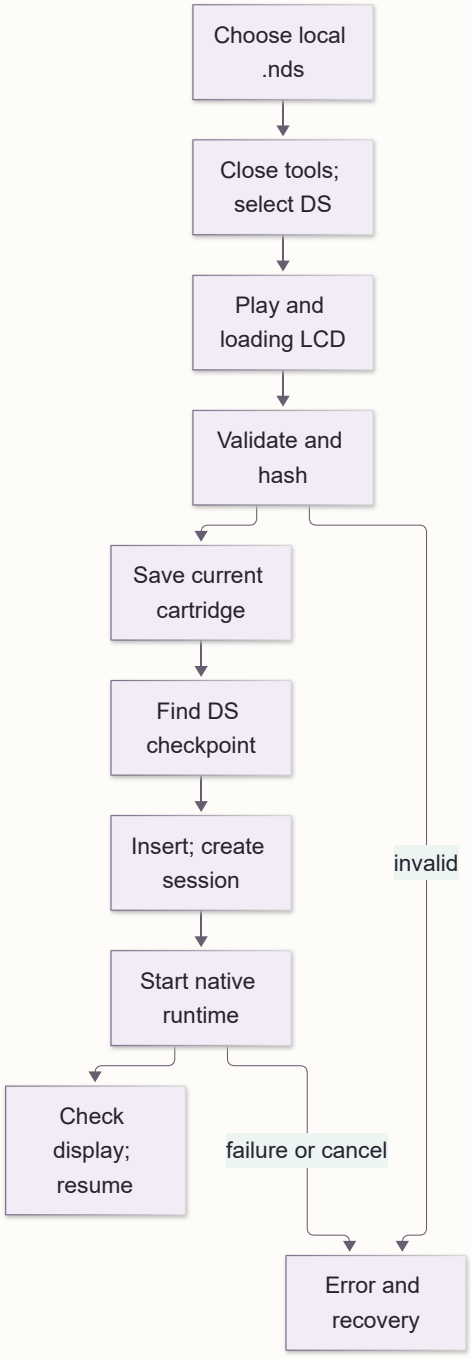


Figure ND04-01. The handoff happens before ROM identification. The loading animation communicates a phase, not measured percentage completion.

Validation has a declared compatibility boundary

`identifyDsRom` accepts an uncompressed `.nds` filename, requires at least 512 bytes and caps the file at 512 MiB. It reads a 512-byte header and rejects nonzero unit-code byte `0x12`, so DSi-enhanced and DSi-only files are outside this implementation's accepted input scope.

For each ARM program descriptor at `0x20` and `0x30`, it checks ROM offset, entry address, load address and length. Program bytes must lie in the file; the load range must lie within the accepted memory ranges; the normalized entry must fall inside that program. These checks catch malformed/truncated files before loading a large native runtime.

They are not a full DS cartridge authenticity or compatibility verifier. A valid descriptor does not prove that a particular game's timing, peripherals or save behavior work in the browser. The full file digest identifies checkpoint ownership; the display title comes from the filename, with underscores replaced and the extension removed.

Two loading visuals solve different problems

The physical cartridge moves from `y = 3.1` toward its seated `y = .44`. The insertion clock accumulates actual painted elapsed time over 1800 ms, using smoothstep `t*t*(3-2*t)` rather than treating a frame count as time. These are local model coordinates, not browser-screen coordinates or a claim about a standard retail shell. Chapter 08 distinguishes the actual modeled slots from the media caption's description of insertion.

The lower LCD separately draws a small original card animation and three progress dots. Its twelve-frame sequence changes on 160 ms buckets. It continues through reading and native startup, then yields to the real framebuffer. It is drawn into a canvas texture rather than fetched as an external GIF, so it remains integrated with the physical display.

With reduced motion, the cartridge is immediately seated and the loading display stays on frame zero. Status still communicates that work is happening. Reduced motion must remove unnecessary motion, not remove the information.

ND04-W01 - Selecting a cartridge with an existing checkpoint

The controller first identifies the file. If another cartridge is running, it saves that session before replacing it. The selected ROM's digest looks up a DS checkpoint. After clearing the previous runtime, the controller stores the new `File`, identity and any battery bytes in memory, begins insertion and launches with the matching checkpoint.

On game start, the adapter loads battery data and native state, then waits for rendered frames before acknowledging readiness. The host checks both native dimensions and the actual retained canvas before replacing loading materials with live outputs. A loaded iframe by itself is not a running DS.

Power is not pause, and resume is not a fresh boot

Pause retains the running runtime and asks it to stop advancing. Switching away from a running, unpaused DS uses `suspend()`: release inputs, save when storage is available, pause and remember that this was an automatic suspension. Returning resumes that retained runtime only if it was suspended. An intentionally paused DS should not become unpaused merely because the player switches consoles.

Power off saves first when running, then clears the runtime, marks power off and installs dark physical screen materials. The selected cartridge stays in memory. Power on launches a new runtime with battery data, not the old native checkpoint. The adapter writes the battery bytes and restarts the core, preserving the distinction between an in-game save and a resumed machine snapshot.

Initial cartridge selection may resume a checkpoint; deliberate power cycling boots from the in-game save. Neither operation persists the ROM itself across a page reload.

ND04-R01 - Storage fails while powering off

`togglePower()` awaits `save()` before clearing a running runtime. If durable storage is unavailable, the save throws with instructions to export. The operation wrapper displays the error and opens tools instead of reporting a successful safe shutdown. The same choice protects against replacing a running cartridge after a failed save.

The application cannot guarantee recovery after browser termination, storage eviction or a user ignoring an export warning. The next chapter on saves explains the portable escape route.

ND04-Q01 - Does a seated cartridge prove that startup succeeded?

Explained answer

ND04-A01 - Animation and execution are separate clocks

Back to question

No. Insertion can finish while the runtime download is still pending. Native success requires a started message and a valid retained display. Conversely, reduced-motion insertion can be instantaneous without altering startup. Treating a visual transition as proof of execution would hide real failures.

Source trail

Cartridge identification and DS options. Controller loading, launch, power, suspension and painting. Adapter restoration and initialization.

PART 2 / MAKING IT RUN

ND05

One framebuffer, two live screens

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND05 / ONE FRAMEBUFFER, TWO LIVE SCREENS

Contents

One framebuffer, two live screens

Question: how do two physical LCDs show one emulator without two cores? **Outcome:** trace the video contract, integer scaling, texture sampling, loading replacement and the distinction between screen brightness and light.

Make the native layout explicit

The native DS picture is two 256 by 192 screens. The adapter configures DeSmuME for top/bottom layout, zero gap and native internal resolution. After startup it checks the core's reported video dimensions: exactly 256 by 384. This prevents a swapped, horizontal or gapped layout from silently producing plausible but incorrect crops.

The actual presentation canvas may be integer-scaled. `dsScreenRegions` accepts widths from 256 through 2048, divisible by 256, with height exactly 1.5 times width. It returns equal upper/lower rectangles. These two checks operate at different layers: native dimensions describe the core; canvas dimensions describe its presentation surface.

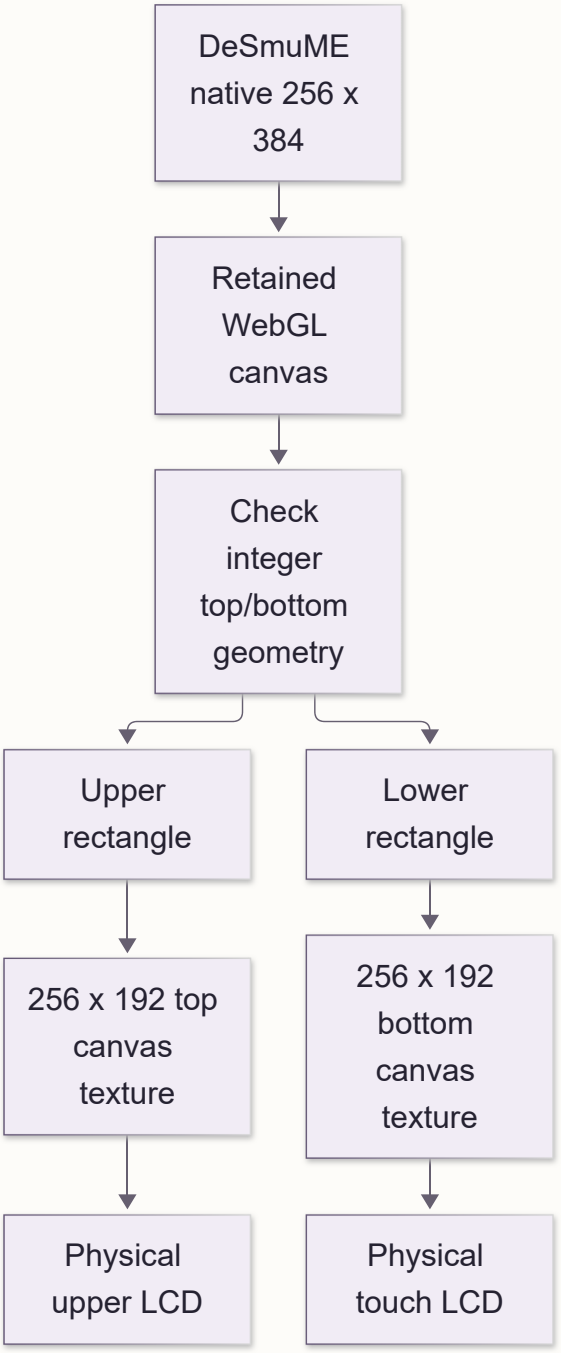


Figure ND05-01. Two views of one running native framebuffer. No second emulator and no independent screen clocks are created.

Retention is an initialization requirement

WebGL does not generally promise that a drawing buffer remains readable after presentation. During runtime setup, the adapter wraps only that core canvas's WebGL `getContext` call to request `preserveDrawingBuffer: true`. On `bba:started`, the host verifies that the actual context reports that attribute. A missing retained buffer fails visibly rather than becoming intermittent black LCDs.

This is a performance trade-off: retention and copying can cost memory and GPU bandwidth. It is not a globally applied WebGL override, and changing the attribute after a context has been created is not equivalent to asking for it when the context is first constructed.

ND05-W01 - An integer-scaled canvas

Suppose the canvas is 512 by 768. Validation accepts it: width is twice 256, height is 1.5 times width. The upper rectangle is `(0,0,512,384)` and the lower is `(0,384,512,384)`. Each is drawn into a separate 256 by 192 output canvas.

`imageSmoothingEnabled = false` avoids an extra 2D interpolation pass. The Three.js maps use nearest minification and magnification filters with no mipmaps. `paintScreens()` marks the maps dirty after copying. These choices preserve crisp source pixels; they do not increase native detail or make a 256-pixel-wide source equivalent to higher-resolution emulation.

The screen is not a lamp-lit plastic surface

Live LCD materials are `MeshBasicMaterial` with `toneMapped: false`, rather than ordinary shell materials whose visibility depends on the desk lamp. The game remains visible when room lamps are off. Shell highlights, surrounding light spill and framebuffer content remain separate concerns.

Before native video exists, status and loading pictures use host-generated canvas textures. `drawLcdCrt` provides their built-in presentation treatment. The controller's DS preferences disable the native shader by default. When the native framebuffer arrives, it replaces the temporary display materials; the implementation does not stack the pre-cartridge treatment on top of the live native picture.

No CRT presentation is proof of timing accuracy. Likewise, a changing pool of light on the desk is not proof that the emulated game accepted input. Pixel evidence, light evidence and input evidence need different checks.

What goes wrong when the contract drifts?

A horizontal canvas can still contain colorful output, but splitting it vertically would duplicate or scramble its screens. A gap changes the touch-to-picture correspondence. A noninteger scale changes the assumptions behind crisp downsampling. `dsScreenRegions` rejects these shapes explicitly.

During screen copying, an unexpected error is reported to the player, the source is detached and tools are opened for recovery. The application does not manufacture a static fallback and call it a live frame.

ND05-R01 - Introduce a 20-pixel gap

A 256 by 404 canvas fails the required geometry. Removing that check would require redesigning both crops and lower-screen coordinate dispatch, not merely tolerating a different height. The safe small change is to retain the pinned zero-gap runtime configuration, not guess the content boundary.

ND05-Q01 - Can two different screen colors establish full DS compatibility?

Explained answer

ND05-A01 - It establishes a narrower fact

Back to question

Different, advancing screen output can establish a functioning split in a particular fixture or game. It does not establish save behavior, native touch positions, every title's timing or performance on phones. Verification must name the input, environment and contract it actually observes.

Source trail

DS options and dsScreenRegions. Framebuffer retention and native checks. Controller message and paintScreens.

PART 2 / MAKING IT RUN

ND06

Touch, controls and the stylus

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Touch, controls and the stylus

Question: why is a DS tap more than a click on a rectangle? **Outcome:** trace a screen coordinate through a 3D ray, UV coordinates, native contact edges and source-owned button state.

Find the visible surface, not an approximate box

The physical DS can rotate, change hinge angle and zoom. An axis-aligned DOM rectangle around its lower display cannot express that perspective or the lid's occlusion. `touchAt(event)` converts the pointer position relative to the renderer canvas into normalized device coordinates. A ray from the actual camera intersects the DS model.

The controller takes the first relevant hit and accepts it only if the object is the lower screen. A hit on the shell or covering lid is not a valid touch. When the held stylus is visible, its own marked geometry is excluded from this search. The pen must not block the LCD it is indicating.

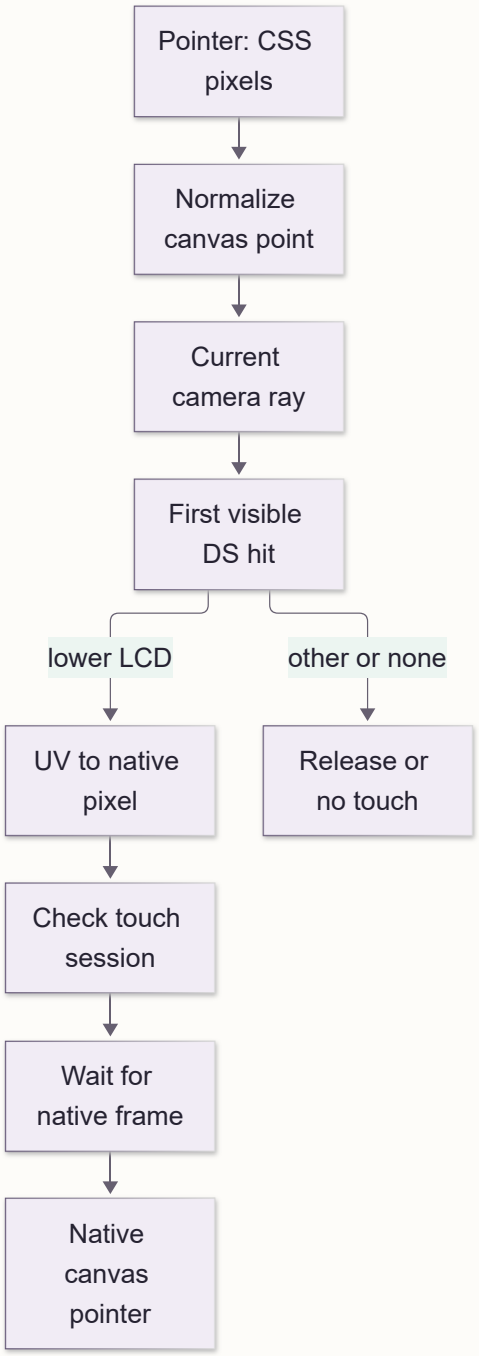


Figure ND06-01. Every transformation has a defined coordinate system. Touch depends on actual visible geometry, not merely a projected bounding box.

UV conversion has a deliberate edge rule

`dsTouchFromUv` rejects missing, nonfinite or out-of-range coordinates. For accepted UV values, it computes:

```
x = min(255, floor(u * 256))
```

```
y = min(191, floor((1 - v) * 192))
```

The vertical flip connects mesh UV orientation to native top-left screen coordinates. The minimum prevents the inclusive UV endpoints from generating out-of-range pixel 256 or 192. Device-pixel ratio does not enter this formula: the DOM event and bounding rectangle use CSS pixels; the ray resolves the physical surface; the UV then resolves native pixels.

The adapter checks integer coordinates again. It maps the native point to the combined canvas using pixel centers, adding 192 to address the lower half. Its dispatch formula accounts for inclusive browser-driver viewport endpoints instead of landing on an offscreen edge.

A fast press and release must survive native polling

Browser tasks and emulator frames are different clocks. Posting down and up in adjacent message tasks can complete before the native core next polls input, producing no observed contact.

`DsTouchQueue` retains contact transitions. After dispatching a sample it awaits `dsInputFrame`, which waits until the core's frame number changes while the queue generation remains current. Consecutive movement samples can coalesce to the latest move; press and release boundaries are retained. The goal is not to replay every mouse move. It is to preserve the semantic edges of contact while keeping movement current.

Cancellation increments a generation, discards pending samples and sends a release when the last dispatched state was pressed. An older asynchronous flush cannot continue injecting a stroke after pause, restore, blur or console switching. This is a cancellation protocol, not a timeout-based guess that the core probably saw a click.

ND06-W01 - A quick tap followed by a drag

Consider samples `down(20,30)`, `up(20,30)`, `down(80,90)`, `move(90,95)`, `move(100,100)`, `up(100,100)`. The two moves can collapse to the latest coordinate, but the first tap's down/up and second contact's down/up remain distinct. The core receives an opportunity to advance between dispatched contact states.

If a modal opens during the stroke, `inputAllowed()` becomes false and input release/cancellation prevents a stuck stylus. A pointer leaving the valid lower-screen surface releases contact rather than drawing through the surrounding plastic.

A physical pen is a separate presentation

The held pen is a clone of the docked model. Its raycast function is disabled and it is added to the scene independently, leaving the original docked geometry in the model's bounds. The held visual points toward the camera with its tip placed on the ray intersection or a camera-facing plane.

The implementation lifts the tip along the pointer ray, not along the LCD normal. Moving along the ray preserves the projected cursor location; moving along the normal could visibly slide the pen tip away from the contact point. Returning the stylus, Escape, right-click, lost availability and blur have explicit cleanup. Visual pickup is not required for direct lower-screen touch.

Buttons have multiple legitimate owners

The adapter holds a `Map` from action to a `Set` of source IDs. Keyboard and pointer can hold the same action simultaneously. Native press state changes only when the set crosses empty/nonempty. Releasing one owner cannot cancel another owner's held direction.

DS keys are arrows for directions, Z/X for A/B, A/S for Y/X, Q/W for L/R, Enter for Start and left Shift for Select. The DS-specific mapping is declared in `DS_CONTROLS`; it should not be inferred from another console's controls. DOM button targets are projected from actual control anchors, hidden when back-facing or covered and disabled outside playing input availability.

ND06-R01 - Blur before the queued release

On blur, release host inputs, return the stylus and cancel native queued contact. Merely clearing the JavaScript array would be insufficient if the native core already saw down. Cancellation must dispatch a release for that last delivered state as well.

ND06-Q01 - Why not coalesce the entire touch queue?

Explained answer

ND06-A01 - Latest position and contact history are different data

Back to question

Replacing all pending samples with the newest one could turn a complete quick tap into an unpressed final coordinate. Coalescing adjacent moves removes redundant positions without erasing down/up edges. That distinction is the central input invariant, and the original Twin Lab fixture tests it through native observations rather than a decorative painted marker alone.

Source trail

UV conversion and DS controls. Controller touchAt, releaseInputs and stylus presentation. DsTouchQueue. Native dispatchTouch and dsInputFrame.

PART 2 / MAKING IT RUN

ND07

Saves, identity and recovery

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Saves, identity and recovery

Question: which kind of progress survives which kind of interruption? **Outcome:** distinguish battery data, native checkpoints, browser storage and portable backups, including the failures each does not solve.

Four objects, not one vague save

An in-game save is the cartridge's battery-backed data, exposed as `sram` by the adapter. A checkpoint also contains the native core's machine state, so it can resume a running situation rather than boot normally. IndexedDB is the browser's origin-local storage location for the latest checkpoint. A `.bbds-save` file is the exported portable envelope containing checkpoint and optional battery data.

These are different layers. A downloaded backup is not automatic cloud sync. An IndexedDB transaction is not an immutable external copy. Loading battery bytes is not equivalent to loading the native state. The ROM is not stored inside either checkpoint or portable backup.

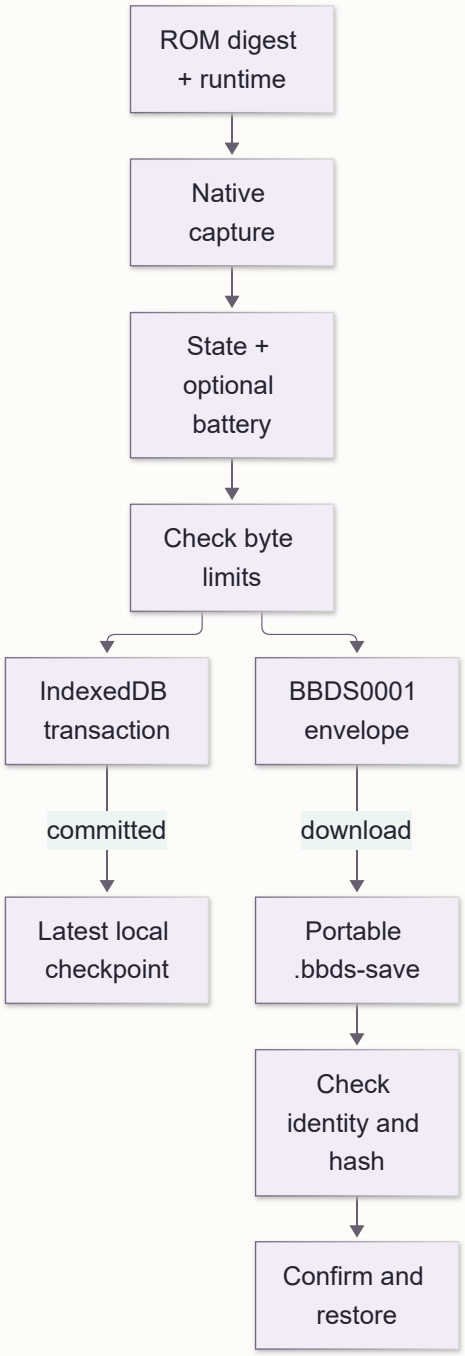


Figure ND07-01. Identity and validation precede persistence or restoration. The two storage routes have independent durability and user responsibilities.

Identity includes the complete cartridge

The host computes the full ROM digest. `DsSaveStore` keys records by `DS_RUNTIME + ':' + digest`, where the runtime marker is `emulatorjs-4.2.3-desmume`. Stored records repeat both runtime and ROM identity, and reads reject incompatible values.

Two files with the same title but different bytes must not share a checkpoint automatically. A title is a label, not a safe identity. Likewise, a future runtime upgrade must not quietly reinterpret old native states. The current marker provides a compatibility boundary; migration requires an explicit future design rather than a renamed string.

The database is `bowserboy-ds`, version 1, with a `checkpoints` object store. It is a separate DS store, not a second view of another console's records. Blocked/open failure is surfaced after a bounded wait. A late successful open after timeout closes its connection instead of reviving an abandoned operation. Version changes close the old connection.

Completion belongs to the transaction

`put` validates capture shape and size, then writes state, battery data, identity and timestamp in one readwrite transaction. The wrapper resolves on `tx.oncomplete`, not merely request success. A transaction can still abort after a request returns successfully.

Native state must be a nonempty `Uint8Array` of at most 64 MiB. Battery data is either `null` or a `Uint8Array` no larger than 16 MiB. `null` means no battery file; an empty byte array is a distinct valid byte sequence. Preserving that distinction matters when reconstructing an export.

Autosave runs every 30 seconds only while running, unpaused, not busy and storage is available. It is not a promise that every last moment survives a crash. The host warns before leaving a running session, but browser termination and eviction cannot be made safe by a warning alone.

ND07-W01 - Reading a portable backup

The first eight bytes are ASCII magic `BBDS0001`; the next four contain a little-endian metadata length. UTF-8 JSON follows, then state bytes and optional battery bytes. Metadata carries runtime, ROM identity, payload lengths and hashes. The battery length uses `-1` for `null`.

The decoder bounds total file size before reading, checks magic, caps JSON at 4096 bytes and checks that metadata fits. It parses JSON, checks exact runtime/ROM identity, validates integer lengths and requires the complete envelope length to match. It then slices the payloads, applies DS capture limits and verifies hashes before returning anything eligible for restore.

```
total bytes = 12 + metadata bytes + state bytes + max(0, battery bytes)
```

This equality rejects both truncation and trailing unexpected payload. Hashes detect accidental corruption relative to the envelope; they are not a cryptographic signature proving who authored the file.

Restore deliberately crosses two boundaries

Import first decodes and validates the backup against the selected ROM. The player confirms replacing the current session/checkpoint. Where storage is available, the host persists the imported capture before requesting native restore. The adapter loads battery data, calls native `loadState` and waits for rendered frames before acknowledging restoration. If the runtime had been paused, it restores that pause afterward.

This is not a distributed transaction between IndexedDB and a native core. An IndexedDB commit followed by a native restore error can leave the imported checkpoint persisted while the live runtime did not complete restoration. The implementation surfaces the error; the book should not invent atomic rollback across those components.

Failed persistence must not look successful

If persistence throws, `storageAvailable` becomes false and the player is told to export before leaving. Saving updates the in-memory battery copy from the native capture, but that does not make the browser record durable. Power-off and cartridge replacement wait for save, so a failed save prevents those normal operations from silently destroying the running session.

Export directly captures the running core and requests a download. It can therefore be useful even when browser storage is unavailable. The application reports a download request, not proof that the operating system saved the file or that a user later tested it on another machine.

ND07-R01 - Rename a ROM and import its backup

If only the filename changes, identical bytes retain the same digest, so identity remains valid. Change one byte and the digest changes; the previous backup must be rejected even when the filename remains identical. This is why a user-visible name is not the storage key.

ND07-Q01 - Is a checksum a guarantee that a backup will always restore?

Explained answer

ND07-A01 - Integrity is narrower than compatibility

Back to question

No. The decoder proves the selected envelope's structural and checksum conditions under the pinned format/runtime identity. It does not prove all future runtime versions, every title's native state behavior, permanent storage or absence of failures during restoration. Exported copies and careful version boundaries reduce risk without removing it.

Source trail

DS capture validation and backup encoding/decoding. DsSaveStore transactions and identity. Controller save, persist and import/export actions. Adapter capture, battery loading and restore.

PART 3

Making it feel physical

1. The DS on the desk
2. Algorithms and data structures
3. Design principles and failure boundaries

PART 3 / MAKING IT FEEL PHYSICAL

ND08

The DS on the desk

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND08 / THE DS ON THE DESK

Contents

The DS on the desk

The DS becomes convincing when its parts obey the same spatial story. A lid must carry its own screen when it folds. A saved playing angle must not erase the owner's tabletop arrangement. A lamp must illuminate the shell from its actual position, not merely make a screenshot brighter. This chapter explains those contracts at source cutoff `61845372142b41992916f3732ea102cc0f4c8edf`. Source inspection here is not a new browser run, physical-device qualification or human visual approval.

One body, two transform spaces

`createDsHandheld()` builds original procedural geometry, not a photograph wrapped around a box. The root owns the lower shell, controls, battery cover, card and stylus. The upper shell and upper LCD belong to a separate `lid` group. Rotating that group changes the upper half without moving the lower touchscreen. The segmented barrel and collars make the joint legible, but the articulation is a bounded transform, not simulated hinge torque.

The two screen planes measure 4.5 by 3.375 model units: the same 4:3 aspect as each 256-by-192 LCD. Surrounds, recesses and controls are separate geometry. That separation matters in Inspect: a screen-sized colored rectangle cannot stand in for a bezel, and a projected button target must follow its real anchor rather than a remembered screen coordinate. See the model construction, screens and hinge.

Rear details are equally structural. Two centered square holes are cut into the extruded outer lid, with inset floors behind them. The supplied DS emblem occupies a lower-body underside plane beside the right battery cover; it therefore does not swing with the lid. Four pale-green square pads, screw recesses and top-edge shoulder housings distinguish the underside from the control deck. Enlarging the emblem preserves its aspect ratio rather than stretching artwork across the battery seam.

There is an important naming boundary around the **bottom-facing slot**. The model's front expansion slot sits along the lower edge of the upright base and rotates its slot face through 90 degrees. It is not the separately named rear DS game-card slot near the hinge. Nor does a README caption saying “slides in from below” establish the card's local-space trajectory: `DsController.load()` starts the card at local Y 3.1 and `render()` seats it at Y .44. These are distinct modeled features, not interchangeable evidence of a hardware connector. The front/underside/card geometry and controller's insertion setup and card animation are the authoritative descriptions.

Pose is a contract, not one rotation

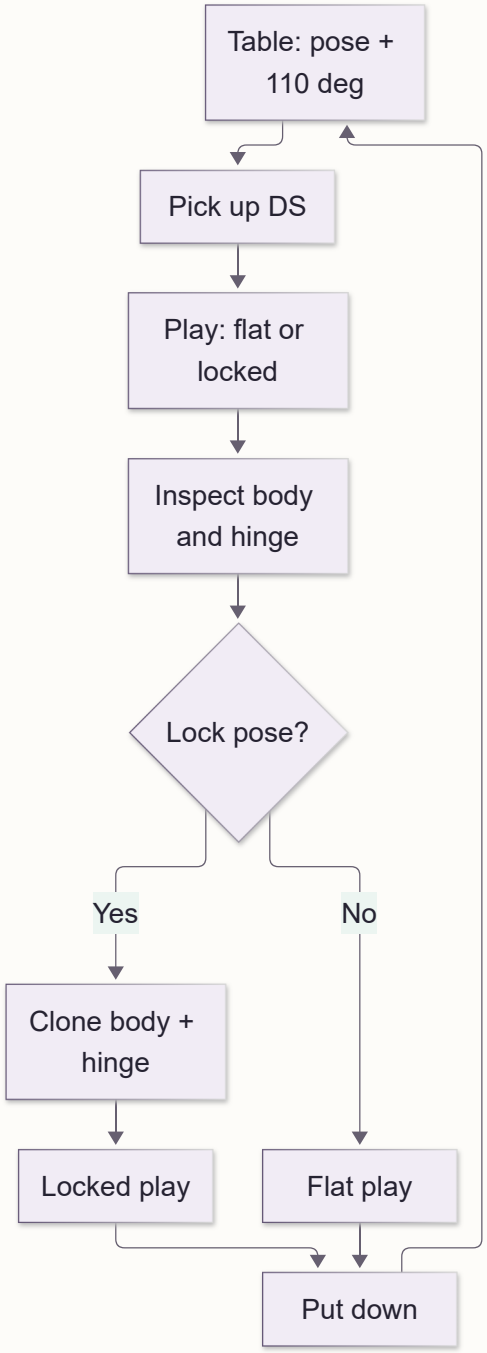


Figure ND08-01. Table placement, Inspect articulation and the session-only gameplay lock are separate state; returning to the table restores its resting hinge.

`setDsHinge()` accepts only finite values and only edits an unlocked DS in Inspect outside a transition. The opening is clamped to 30–180 degrees; the stored lid rotation is `180 - opening`. Thus 180 degrees means zero lid rotation, while the table's 70-degree rotation represents a 110-

degree opening. Confusing opening angle with stored rotation reverses an otherwise plausible slider.

`setDsPoseLocked()` clones the root quaternion and stores the lid rotation. It also clears active drag/pinch state, preventing an old gesture from continuing across the lock boundary. `setView()` chooses the saved desk rotation for table mode, the current rotation for Inspect, or the locked rotation/default identity for play. A lock survives view changes within the session; reloading is not pose persistence.

The camera uses the actual articulated bounds, not a flat-console rectangle. `dsPoseBounds()` evaluates body and lid geometry under their respective candidate transforms. `updateCameraTargets()` lifts the resulting envelope above the desk and fits its width, height and depth, reserving additional space for the phone Inspect panel. Immediate matrix updates keep projected controls consistent before another eligible GPU paint. These mechanisms are visible in pose editing and bounds, camera fitting and view transitions. Fitting is not visibility: an extreme backward pose can fit perfectly while the lid hides a screen or control.



Figure ND08-02. Existing DS Inspect GIF. The book renderer embeds only its first-frame PNG still, not the hinge motion; the repository GIF retains the sequence. LCDs replay a recorded native opening, not responses to these scene actions. Print still: frame zero of the animated source; not motion evidence.

Clearance follows the interaction

The desk is not organized by hiding inconvenient objects. In focus mode, the active DS is an immovable obstacle for placement purposes: its playing angle must not yield to a decorative prop. The inactive console is placed next, followed by Pear and the axolotl, then loose desk objects. In table mode, companion positions receive priority before props and consoles. Returning from focus restores remembered companion XZ coordinates, then reconciles lower-priority objects.

`resolveDeskClearance()` first accumulates candidate placements. If any placement fails, it reports blocked space without applying that partial list. On the table it retries the two console orders, because greedily placing the narrower object can trap the wider one. A signature avoids repeating unchanged work. This is deterministic bounded rearrangement, not a globally optimal packing solver.

`nearestDeskPosition()` reduces vertically overlapping obstacle boxes to forbidden XZ rectangles expanded by the moving object's bounds and a .25-unit gap. Candidates lie at the desired coordinate or rectangle edges, clamped to the workspace. It chooses the nearest valid squared-distance candidate; a small floating-point tolerance keeps a legal edge legal. Conservative axis-aligned boxes can reject visually empty corners around a rotated DS, trading packing density for predictable separation. Read mode priorities and atomic application with the placement solver.

Light must move through real space

The metallic lid uses blue `#0077FF`; the lower shell uses lighter `#40A0FF`, with grain, roughness and clearcoat. Those values do not promise the same perceived blue under every warm source. Brightening a material to compensate for backlighting would change other views too.

Workstream 116 records the causal correction: the old upright target `(5, 5.1, 8)` stood ahead of the lamp's emitting region. A first candidate `(-2, 5.1, -2)` improved illumination but failed a rotation-clearance check. The final shared target is `(-2, 5.1, -8)`. Moving the object rather than adding a camera light preserved the existing paint, exposure and dark-room behavior. These measurements are historical task evidence, not measurements repeated for this chapter.

Pear and the axolotl are light-bearing scene objects, not glowing stickers. The axolotl's point light is a child of its movable root, so its light pool follows translation. Scene setters change emitted material intensity and light intensity together; off means removing illumination, not merely darkening a button. `DeskMotion` supplies held/released motion, gravity and damped impacts. Pear adds expressive recovery; the smaller companion reuses the motion without pretending to share every facial animation. See companion brightness, axolotl light attachment and drop integration.



Figure ND08-03. Existing DS lighting GIF, represented by a first-frame PNG still in the rendered book. The still cannot demonstrate switch transitions or establish lighting-independent paint fidelity. Print still: frame zero of the animated source; not motion evidence.

ND08-W01 — Work a pose change through the system

Starting in unlocked play, enter Inspect and set the opening to 120 degrees. The lid rotation becomes 60 degrees. Rotate the body, lock, then return to play: the quaternion and hinge should be retained while camera fitting uses the new envelope. Put down: the desk orientation and 110-degree opening return. Pick up again: the session lock supplies the playing pose.

This is a reasoned walkthrough, not a claimed execution. Its diagnostic value is separating three failures: wrong hinge conversion, overwritten desk placement, and camera framing based on stale bounds.

ND08-R01 — Retry with a changed condition

Now put a companion where the DS envelope needs clearance and repeat the transition. The focused DS should keep its pose while lower-priority placements yield. If no placement exists, spread objects out or reset a clear home; do not shrink the DS, disable depth testing or silently discard the lock. Then retry with every lamp off. Darkness is expected, not evidence that the corrected material reverted.

ND08-Q01 — Why not always move the DS to resolve a collision?

Explained answer

ND08-A01

Back to question

Because focus mode promises a stable chosen pose and usable projected controls. Moving that priority object would make a decorative collision change the user's playing composition. Table mode has different ownership and may legitimately relocate consoles. Neither policy guarantees that every crowded arrangement has a solution.

Historical evidence and limits

Public workstreams 85, 87, 93 and 116 preserve the placement/pose failures and correction. 95, 100 and 103 explain rear refinements; 110 and 111 explain the second movable lamp. Checked implementation tasks and local captures do not erase unchecked hosted qualification or owner-review items. Public delivery proves delivery; a human must still judge the physical composition. No new capture, artwork or approval is supplied here.

PART 3 / MAKING IT FEEL PHYSICAL

ND09

Algorithms and data structures

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Algorithms and data structures

Question: which small algorithms make this appliance coherent? **Outcome:** reason about time, coordinates, ownership and memory without mistaking a familiar data structure for an automatic performance guarantee.

Complexity starts with a declared unit

Let `R` be ROM bytes, `S` native-state bytes, `B` battery bytes, `q` pending touch samples, `h` physical button actions and `p` active command requests. Costs below describe application-side work, not the native emulator's CPU accuracy or instruction throughput. Hashing and copying do not become free because their APIs are asynchronous.

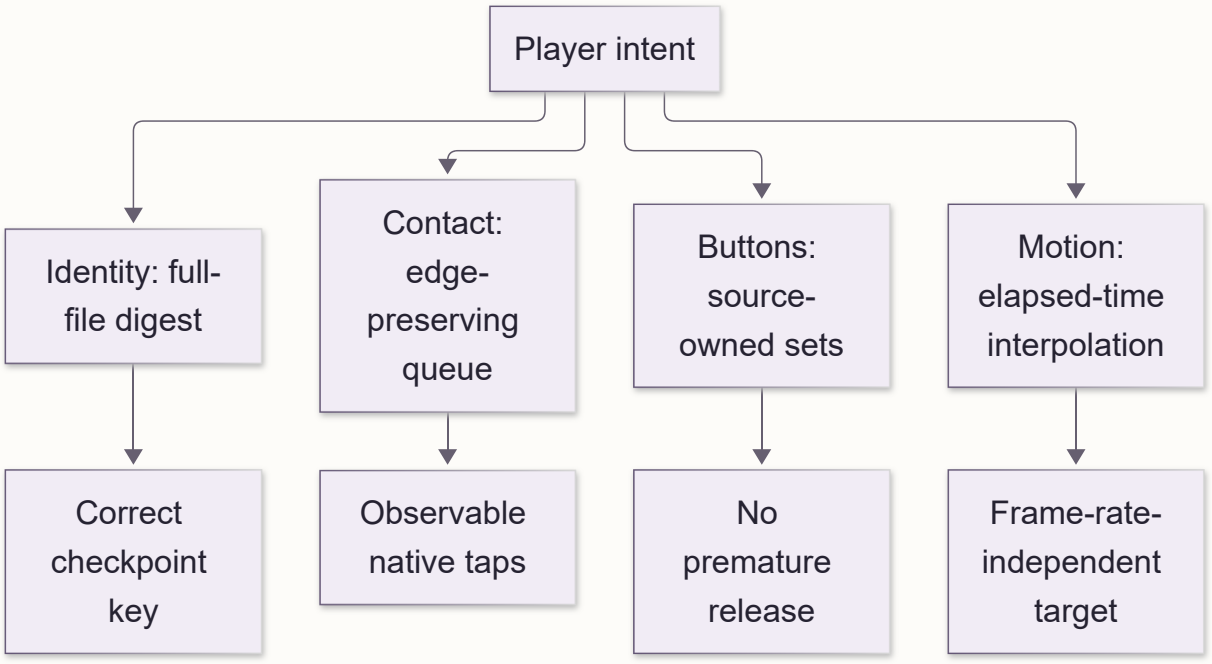


Figure ND09-01. Four different invariants. Each algorithm protects a specific meaning, not a generic promise that the entire emulator becomes fast.

Full-file identity: pay once per selection

Cartridge identification reads and hashes the entire `R`-byte ROM after checking the header. Work is linear in file size. An `ArrayBuffer` for the file means application memory proportional to `R`; native loading and structured cloning may add further live copies or allocations. The book does not claim zero-copy loading or bound total memory by the ROM alone.

That cost buys exact byte identity rather than a heuristic such as title plus file size. The early scene handoff hides neither the cost nor errors: it gives the player a meaningful display while awaited work proceeds.

A queue preserves edges, not every pointer coordinate

`DsTouchQueue.samples` is an array. A new move can replace a pending move at the tail. Contact starts and ends append separately. A generation counter invalidates in-flight work during cancellation. Dispatch is serialized by `flushing`, and each delivered sample awaits native frame advancement.

Tail replacement is constant-size work; array `shift()` conventionally moves remaining indexed entries and can cost O(q). The important practical limit is that high-frequency adjacent moves coalesce. A long sequence of alternating taps is not given a strict fixed queue bound by this code. Calling it a constant-memory queue would be inaccurate.

A future ring buffer could improve dequeue behavior if measured workloads justify it. Such a change would still need to preserve contact edges, cancellation generations and error propagation. Data-structure substitution alone does not prove the semantic invariant.

ND09-W01 - Why a boolean cannot represent a button

Keyboard Right goes down, then a pointer presses Right. There are now two source IDs in the action's set. The native button was already down, so no second native edge is needed. Keyboard release leaves one source; native Right stays down. Pointer release empties the set and sends native up.

The adapter's `Map<action, Set<source>>` models exactly that ownership. Under conventional hash-table behavior, lookup/add/delete are expected constant time; worst-case engine guarantees should not be inferred from that notation. Releasing everything visits the actual stored sources.

One boolean would fail when the first owner released. A reference count would handle balanced edges but would be harder to make idempotent under duplicate source events. Source sets make ownership explicit.

Maps also correlate commands

`pending` maps each request UUID to callbacks and a timer. Results locate their own request rather than relying on message arrival order. Timeout removes the entry before rejecting. Session teardown clears timers, rejects every remaining promise and empties the map.

The map uses O(p) entries, not unlimited history. The controller's busy guard serializes ordinary operations, but the data structure itself is not a formal hard cap on all possible outstanding requests.

Coordinates: affine conversion followed by geometry

The pointer is normalized within the canvas to `[-1,1]` in X and Y. Raycasting resolves the first visible surface under the current camera. An accepted lower-screen UV becomes a native integer pixel with vertical flip and inclusive-edge clamping. The adapter then maps that pixel's center into the lower half of its combined canvas.

The scalar conversions are O(1). Ray traversal depends on scene geometry and Three.js behavior; the book does not claim that intersecting the model is constant time. More importantly, replacing that geometry query with a rectangle changes semantics, not just cost.

Elapsed-time motion and damped response

Insertion accumulates elapsed paint timestamps and uses smoothstep `e(t) = t²(3 - 2t)` for `t` clamped to `[0,1]`. At `t=.5`, `e=.5`; its endpoint slopes are zero, making the approach gentler than a constant velocity jump. A missed paint produces a larger next elapsed step rather than assuming a fixed number of frames.

D-pad pitch/yaw use signed opposing directions. Diagonal magnitude is normalized by `max(1,hypot(x,y))`, so holding two perpendicular directions does not multiply total tilt by square root of two. Exponential damping uses the measured delta and separate press/release response rates; tiny residual differences snap to target to stop unnecessary repainting.

Backup validation has linear payload costs

Portable decoding checks size and metadata first, then copies and hashes payloads. Work and memory are proportional to `S+B`, within the declared limits. A checksum is O(payload), not a constant-time header check. The bounded JSON header avoids trusting an arbitrary metadata length before basic envelope validation.

ND09-R01 - Increase display refresh without changing motion speed

At twice the browser paint frequency, a frame-count-based insertion finishes twice as fast. Elapsed-time accumulation retains the 1800 ms target, subject to when the browser actually paints. It cannot guarantee that suspended tabs render motion smoothly; it guarantees the target is expressed in time, not an assumed frame rate.

ND09-Q01 - Which optimization is safe without additional evidence?

Explained answer

ND09-A01 - None merely because it sounds cheaper

Back to question

Replacing SHA-based identity with a title, touch edges with a latest-only sample or geometry with a bounding box changes correctness contracts. Measure costs first, preserve the invariant in focused tests and name any new compatibility boundary. Simpler code is valuable only when it preserves the meaning the current algorithm protects.

Source trail

Cartridge identity, coordinates and backup algorithms. Edge queue. Source-owned inputs. Controller command and motion algorithms.

PART 3 / MAKING IT FEEL PHYSICAL

ND10

Design principles and failure boundaries

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Design principles and failure boundaries

Question: which principles actually explain this implementation? **Outcome:** recognize useful boundaries and their limits without inventing a perfect architecture or attaching pattern names to every function.

Own the truth at one layer

The host owns checked storage and player-visible lifecycle. The adapter owns version-sensitive native calls. The physical model owns geometry and anchors. The native core owns emulated state and output.

This separation prevents two persistence engines from independently choosing which checkpoint to resume. It also keeps a scene transformation from becoming a cartridge reboot. The implementation is not perfectly decoupled: the controller directly reads the same-origin canvas, and projection requires knowledge of scene objects. Those are deliberate integration boundaries, not evidence that every dependency has disappeared.

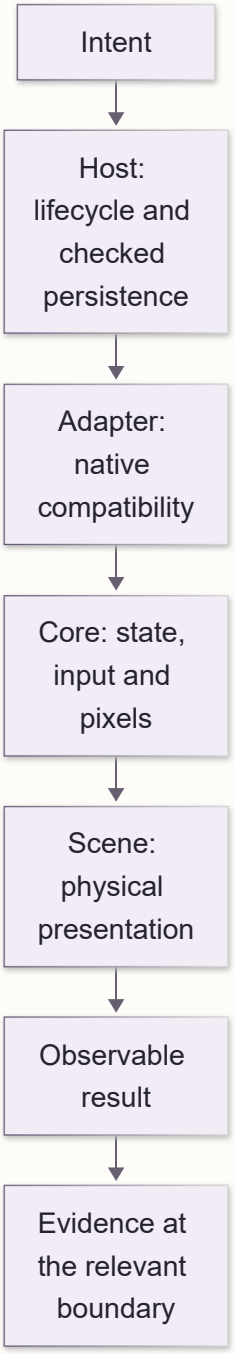


Figure ND10-01. Ownership follows the operation, then verification follows its observable consequence. A source change alone does not close the final arrow.

Fail early where the assumptions become knowable

Header ranges can be checked before native startup. Session identity can be checked when a message arrives. Video layout can be checked only after the core exposes dimensions. Storage durability can be acknowledged only when its transaction completes.

Moving a check earlier is useful only when the necessary information exists. For example, an `.nds` extension cannot establish that the ARM program bytes fit in the file; a valid header cannot establish native video readiness. The design uses a chain of narrower checks rather than one mythical “validated ROM” boolean.

ND10-W01 - A corrupted checkpoint must not become a fresh start

Suppose a matching IndexedDB record contains invalid state bytes. A success-shaped fallback would silently ignore the record and start a new game. That would make the app appear operational while concealing lost progress.

The store instead validates identity and capture shape and throws on an invalid record. The operation wrapper reports the error and opens tools. The portable decoder likewise rejects a bad checksum before restoration. The visible result can be less convenient, but it does not pretend the requested recovery happened.

Preserve invariants through every entry point

Keyboard, pointer and runtime-focused input share source-owned button semantics. Pause, configure, restore and hidden visibility cancel native touch. Power off, replacing a cartridge and switching consoles must release inputs before changing the runtime's availability.

A feature that works through one button but leaves a second entry point with stale state is incomplete. This is why a stylus demonstration needs cancel/blur cases and why visual model rotation must be tested alongside touch coordinates, not as an unrelated aesthetic action.

Cancellation is a first-class state transition

The touch queue's generation distinguishes current work from stale asynchronous work. Launch cancellation rejects a pending launch callback. Session clear rejects outstanding commands. These mechanisms do not silently convert abandonment into success.

They also have different scopes. Cancelling native startup is not the same as aborting an already awaited full-file read. Invalidating queued touch is not the same as terminating the native runtime. Good documentation states which operation is cancellable instead of generalizing one Cancel button to the entire application.

Separate simulation, presentation and observation

A physical button can move without native input being accepted. An LCD can display replay footage without a running core reacting. A lamp can change the shell while the self-lit LCD remains readable. A passing fixture is not every commercial game's compatibility.

The book's source-backed diagrams explain implementation. The visual showcase demonstrates scene presentation with recorded opening replay. The original Twin Lab tests observe native contracts. Release receipts qualify a specific revision in a specific environment. None substitutes for all the others.

ND10-R01 - Upgrade the runtime

Treat an upgrade as a compatibility change, not a URL edit. Recheck game-manager APIs, framebuffer retention, dimension/layout settings, native touch polling and save-state behavior. A changed native state format may require a new runtime identity and explicit recovery guidance.

Reusing old hashes or old checkpoint metadata would not establish new compatibility. Preserve the old book and release evidence as dated records, then qualify the changed implementation independently.

ND10-Q01 - Does isolation mean that switching consoles cannot fail?

Explained answer

ND10-A01 - A boundary reduces coupling, not all failure

Back to question

Separate sessions and stores prevent accidental shared-state ownership. Switching can still fail while saving, pausing, starting or accessing the CDN. The controller must report those failures and retain useful state where possible. Independence is an architectural property; successful switching is an observed operation with its own prerequisites.

Source trail

Controller lifecycle, errors and cleanup. Native adapter boundary. Checkpoint identity and transaction ownership. Cancellation generation.

PART 4

Evidence and maintenance

1. Tests, releases and honest evidence
2. Visual documentation
3. The maintainer field guide

PART 4 / EVIDENCE AND MAINTENANCE

ND11

Tests, releases and honest evidence

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

Tests, releases and honest evidence

What exactly became public?

A passing local DS check, a green hosted workflow and a byte-matched public site answer different questions. This chapter's motivating question is how to report a release without collapsing them into “verified.” Its outcome is a reproducible interpretation of the cutoff receipts, including failures, skips and the public verifier's remaining exception.

At exact cutoff `61845372142b41992916f3732ea102cc0f4c8edf`, Pages **42390309869** succeeded with **119 declared / 98 PASS / 21 SKIP / 0 FAIL / 0 UNRUN**, across all 19 browser shards, build and deployment. The preceding implementation recovery, `387537b602f44a5bc85d10ee0196d0ccb099e968`, Pages **42389431053**, has the same counts. The latter is recorded in pinned documentation; the final documentation-only cutoff run is identified by its actual hosted logs. [S1, S2]

These are historical hosted results inspected for this manuscript. No new application tests were run to author the chapter.

Keep four evidence layers separate

Local implementation evidence includes units, syntax/build checks and focused browser cases. For immediate DS loading, the public TODO records held upload reading, native handoff, invalid replacement retention and cancellation. Upright lighting adds real shell-ray samples and a bounded clearance correction. Neither local record alone says that the exact source was deployed. Their TODOs explicitly separate technical completion from publication. [S3]

Hosted qualification establishes what ran in the workflow environment at a specific SHA. Inventory, shard summaries and skipped declarations matter as much as an overall green status. A scenario that never ran after a shard exhausted its budget is not the same as a skipped case. A case that passed on a previous source is not evidence that it passed on the next one.

Actual public evidence checks the delivered site or repository media. Successful upload/deployment jobs do not prove that the browser fetched the intended bytes. Workstream 112 records independent runtime/entry/demo comparisons and actual live-site cases after the monitored release. [S4]

Acceptance and compatibility remain separate again. Current supported DS status came through workstream 114; it did not certify all games. The direct-boot guide still limits its BIOS-free demonstration to Twin Lab rather than every accepted original-DS ROM. Human aesthetic approval and physical-phone qualification cannot be inferred from these technical layers. [S5]

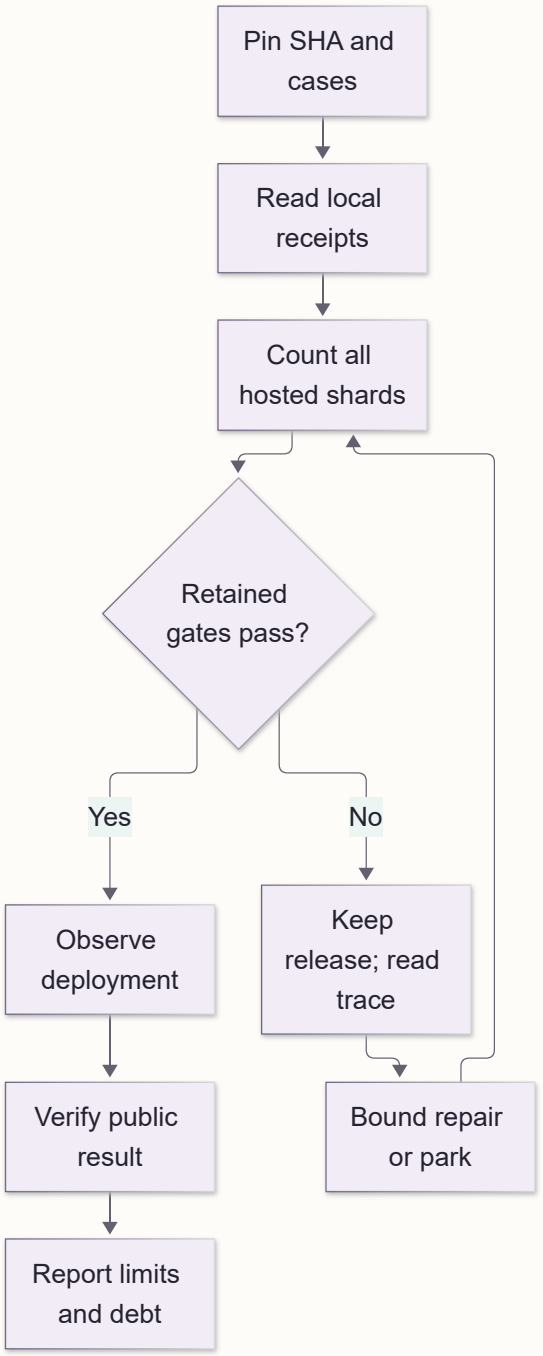


Figure ND11-01. Release evidence is a chain of independently bounded claims; a later link does not erase an earlier failure.

The first recovery disproved the easy environment explanation

Workstream 104 began with nine hosted failures and deployment skipped. Selecting explicit full-resolution ANGLE SwiftShader was a bounded environment hypothesis, not a confirmed diagnosis. Its exact hosted attempt produced 52 passes, 55 failures and two unrun cases: a

regression, not a repair. The launch selection was restored rather than presented as a success. Loader-aware setup and a directly coupled DS fill correction were then qualified without blanket-skipping the newly failing cases. [S6]

The next run reached 90 PASS / 7 FAIL / 10 SKIP / 2 UNRUN. Inspection of all 19 shards found suite exhaustion affecting isolation and resize. Those unrun scenarios were preserved and native cases redistributed. The eventual 92 PASS / 17 SKIP / 0 FAIL / 0 UNRUN recovery executed both formerly aborted cases successfully. This was a stronger result than making the totals green by hiding unexecuted scenarios. [S6]

The debt subsequently grew through explicit bounded decisions. A companion lamp obstructed a console-movement fixture; a complete reduced-motion/ WebGL scenario remained unqualified after its repair; GPU-fence and tripod cases failed their allowed follow-up. At cutoff the partition is **18 CI-only skips and three also skipped locally**, totaling 21. [S7]

Local focused passes do not restore those exact hosted cases. Workstream 105 requires original assertions, resolution and deadlines on the hosted browser before removing a skip. Its inventory includes DS native boot, quick taps, side-stylus, hinge, placement, reference lighting, shoulder motion, several stylus poses and independent LCD spill. The honest supported-console statement must coexist with these qualification gaps.

ND11-W01 - Account for the fullscreen recovery

The untouched-opening media revision `c3b211e` had public GIF evidence but did not deploy through Pages 42388336737: 119 declared, 97 PASS, 21 SKIP, one FAIL and zero UNRUN. Its fullscreen-return trace showed the final changed LCD hash completing just after the original 120-second deadline. That is not proof of a stuck LCD or a DS-core defect. [S1, S8]

One bounded **test-only** correction disabled unrelated Pear and axolotl companion lamps through existing scene methods. The source change is visible in the shared fullscreen fixture. It retained both exit paths, viewport, renderer resolution, deadlines and original runtime/pixel/input assertions. The exact local case passed in 30.8 seconds without retry; the exact recovery then passed hosted and deployed. [S1, S9]

This shared release gate belongs here because it governed publication of the DS-bearing site, not because it supplies new DS-specific fullscreen compatibility evidence. Its isolation repair must not be described as a shipping lighting optimization or a repair of all slow-rendering cases. The count stayed at 21 skips: this failure was repaired without another park, and the previous debt remained untouched.

ND11-R01 - Retry once, then preserve the unresolved question

The recovery policy is bounded by the observed case, not an invitation to retry unrelated failures indefinitely. Inspect the exact trace, identify the fixture or product assumption being changed, and preserve the original question. Workstream 112 illustrates the distinction: opening recovery

controls needed a real product correction after the dock became default-collapsed, but a focused recovery-entry pass did not restore the complete parked reduced-motion/WebGL case. [S4]

For the fullscreen follow-up, another failure would have meant explicit parking under 105 rather than another repair loop. Hosted success made that contingency unnecessary. Neither the successful repair nor the subsequent receipt-only cutoff commit changes what the parked cases mean.

Public bytes and public pictures prove different things

For the preceding `387537b` release, all **48 actual runtime/entry/demo assets matched** committed inputs. The strict additional `.nojekyll` check returned **404**, leaving the 49-check verifier at exit 1. Report exactly that: 48 matches and one known marker exception, not 49 passes or a wholly green strict verifier. The pinned receipt describes the marker as build-only; do not silently delete it from the result. [S1]

Media needs a parallel limitation. Workstream 117 records live native interaction captures; workstream 120 deliberately replaces scene LCD content with capture-only replay of untouched openings. Real scene controls plus recorded LCD playback do not demonstrate live game-input response. Byte-identical public GIFs establish delivery of those GIFs, not universal compatibility, audio quality or physical-phone speed. [S8]

ND11-Q01 - Exercise: rewrite an overconfident release summary

Explained answer

“119 tests passed, all 49 public checks passed, and the new footage proves native DS input.” Correct each claim using the cutoff boundary.

ND11-A01 - Explained answer

Back to question

Write: “119 cases declared: 98 PASS, 21 SKIP, zero FAIL and zero UNRUN in Pages 42390309869. The preceding recovery's public receipt records 48 asset matches and `.nojekyll` 404, so the strict verifier exits 1. Recorded-opening scene media demonstrates scene choreography and delivered footage, not live native input.”

The qualification about the preceding public receipt is intentional: the cutoff workflow was inspected, but this authoring task did not perform a fresh cutoff-SHA public byte comparison. Precision about that boundary is more useful than manufacturing one last all-green sentence.

Source references

- **S1:** Previous recovery, exact counts and strict verifier exception, verification lines 14–37.

- **S2:** Exact cutoff Pages 42390309869: actual head SHA, successful build/19 shards/deploy and shard-log totals inspected during authoring. This post-commit run cannot be recorded inside its own pinned source tree.
- **S3:** Local DS loading receipts, TODO 115, lines 25–37; lighting and clearance receipts, TODO 116, lines 25–46.
- **S4:** Monitored/public evidence, TODO 112, lines 27–49; recovery-entry versus parked-case distinction, lines 76–98.
- **S5:** Current console status, TODO 114, lines 15–34; direct-boot limits, guide lines 177–193.
- **S6:** Failed environment hypothesis, shard exhaustion and eventual recovery, TODO 104, lines 26–72.
- **S7:** Exact restoration contract and parked inventory, TODO 105, lines 8–36; growth to 21, lines 80–108.
- **S8:** Live capture qualification, TODO 117, lines 15–43; replay boundary and release receipts, TODO 120, lines 17–65.
- **S9:** Actual bounded lamp isolation, fullscreen fixture lines 915–931; causal ledger, lines 17–37.

PART 4 / EVIDENCE AND MAINTENANCE

ND12

Visual documentation

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

ND12 / VISUAL DOCUMENTATION

Contents

Visual documentation

A DS showcase must make two screens, a hinged body and a small set of physical interactions understandable before a reader studies the guide. It must also say what was actually recorded. A beautifully timed button press beside moving LCD pixels can suggest a causal relationship that the capture does not contain. Good visual documentation controls that interpretation as carefully as it controls framing.

This chapter uses the public repository at `61845372142b41992916f3732ea102cc0f4c8edf`. It explains the existing workflow, not a new capture session. Private cartridges, videos, timing receipts and browser stores were not needed to write it and are not book inputs.

The README is a front page, not the entire manual

The repository's GIF-first policy is explicit and local to this project. The front page offers motion and short captions; setup, detailed controls, limitations and release evidence live in linked documentation. This division is especially useful for DS: a hinge sequence communicates articulation quickly, while the 30–180-degree limit, session-only lock and occlusion rules require prose that should not interrupt every animation.

The documentation chain therefore has different jobs. The README attracts and orients. The guide tells a player how to act. `MEDIA.md` establishes capture provenance and interpretation. Public TODOs retain causal failures, acceptance criteria and qualification boundaries. This engineering book explains why those choices exist. Copying all five layers into the front page would make each layer harder to maintain.

The approved DS logo is artwork, not a native-game screenshot. Likewise, an image of a rear emblem documents branding placement, not cartridge compatibility. Workstreams 92 and 99 concern DS visual identity; 95, 100 and 103 concern physical markings. Keeping those claims separate prevents an attractive identity treatment from becoming accidental runtime evidence. The controlling policy and current edition are recorded in MEDIA.md, opening edition.

Two timelines, one scene

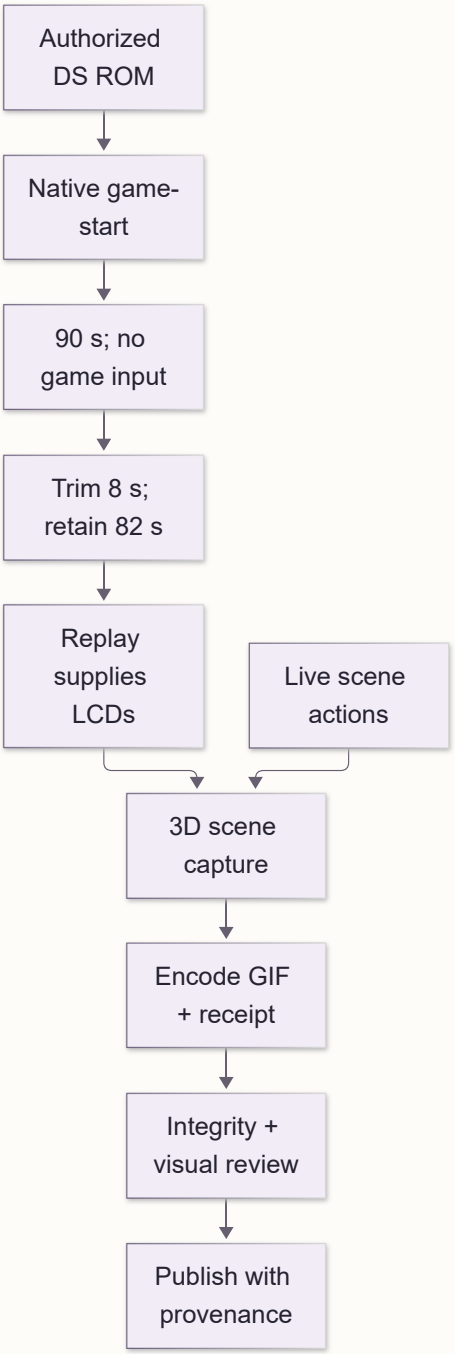


Figure ND12-01. Native LCD acquisition and live scene manipulation meet only in the capture harness; their coexistence does not prove that the game responds to the depicted controls.

The current edition records the selected DS opening for 90 native seconds, then excludes its first eight seconds. Native time begins at entry to `EJS_onGameStart`, after core/cartridge loading, not at page navigation and not at a later host-ready notification. The recorder intercepts the served

player source in an isolated browser context and captures the native canvas. The DS row explicitly selects 90 seconds and an eight-second trim. See native acquisition hook.

That recording then becomes a source for a second operation. `openingReplay()` serves private opening videos through intercepted browser routes, creates off-DOM video/canvas pairs, and exposes the DS canvas through an instance-specific `source` accessor. Browser actions still operate real scene controls; the screen image is supplied by the recording. Geometry, materials and lighting are not replaced with a composited illustration. The implementation boundary is explicit in the replay installation and receipt declaration.

Consequently, a DS button GIF establishes physical button travel while an untouched opening plays. A stylus GIF establishes pickup and scene tracing, not native touchscreen response to that trace. A lighting GIF establishes scene light changes with recorded LCD content. Those are useful, honest demonstrations. Calling them live game-input demonstrations would be false, even though the source pixels originally came from a running native core. Capture-only replay must not become a shipping-player fallback.



Figure ND12-02. Existing DS opening GIF: the selected native opening's 8–90-second window, without game inputs. The book renderer embeds only a first-frame PNG still; sequence duration and pacing remain properties of the repository GIF, not this printed frame. Print still: frame zero of the animated source; not motion evidence.

Why an earlier honest capture can still be superseded

Workstream 117 produced fresh DS and shared-lighting recordings. Its capture state was not fabricated, but the owner rejected the tutorial/menu screens and requested untouched openings throughout the README. Workstream 120 therefore replaced every scene GIF, including desk and lamp sequences, rather than fixing only the main DS introduction.

This distinction is a maintenance lesson: technical authenticity does not guarantee that a capture satisfies the requested editorial state. Keep the earlier edition labeled historical; do not silently reuse its native-input description for replacement replay footage. `MEDIA.md` explicitly separates the current opening edition from the superseded capture round.

Loading is another editorial hazard. Workstream 115 moves selection back to the physical DS immediately and supplies animated loading feedback before slow identification/startup finishes. A short loading clip is not a boot benchmark. Cutting before readiness is acceptable if the caption says the full boot takes longer; implying that a cartridge starts within the clip's duration is not. The same discipline applies to workstream 116: an upright lighting clip documents the real lamp-pool placement, not a new studio rig or a promise that metallic blue is invariant under warm light.

Reproducibility without publishing private inputs

Reproduction scripts belong in the repository; local source recordings, ROMs and detailed receipts do not. Environment variables choose authorized local sources, and the capture receipt records revision, viewport, clip boundaries, errors and replay observations. This chapter references those public mechanisms without opening private evidence or copying personal paths. A future maintainer must supply their own lawful inputs and explicit capture destination; reading these instructions does not authorize a new capture or publication.

`capture-desk-showcase.mjs` measures clip starts and durations and records LCD progress at their boundaries. Its DS Inspect sequence exercises body, hinge and lock controls; lighting sequences operate actual scene switches. The final receipt retains errors rather than treating an incomplete take as an ordinary clip. See clip bookkeeping, DS scene sequences and receipt output.

Encoding is receipt-driven, not a manually guessed crop. `clips()` rejects capture errors and missing durations, validates frame rate and palette size, then uses FFmpeg scaling, palette generation and Bayer dithering. Ordinary feature clips trim half a second from the tail. Complete showcases provide `encodedDuration`, bypass that trim and calculate the final GIF delay in centiseconds. The complete DS clip uses eight frames per second and 64 colors to retain all 82 seconds below the publication size limit. Reduced palette and frame rate are delivery tradeoffs, not higher-quality evidence. The encoder and complete-window recorder explain that distinction.



Figure ND12-03. Existing shared-desk GIF, relevant to DS scene clearance and moving illumination. Only a first-frame PNG still is embedded by the book renderer; it cannot establish the drop trajectory or moving light pool. Visible LCD content is recorded replay. Print still: frame zero of the animated source; not motion evidence.

Four different kinds of evidence

Local integrity answers whether files decode, have expected dimensions, durations and loop flags, and differ across temporal samples. A changing sample does not show that every frame is correctly framed.

Bounded visual review answers whether inspected frames show the intended state. The recorded workflow caught an axolotl panel hiding its lamp and a settings sequence missing its drawer. Replacing those takes addressed specific observed failures, not every possible visual defect.

Hosted qualification answers whether the exact source passed its delivery gate. `MEDIA.md` records that the opening-media commit's Pages run failed an unchanged fullscreen case and skipped deployment, followed by a bounded fixture correction and successful later deployment. Public GIF byte checks were a separate result. A reachable GitHub image did not retroactively turn the failed Pages run green.

Human approval concerns whether the owner accepts the appearance and editorial choices. Neither an empty browser error array nor a successful deployment supplies that judgment. The historical successful run still retained 21 existing skipped browser scenarios; this chapter creates no new test result or compatibility claim. See the publication record.

ND12-W01 — Write the caption from the evidence

Suppose the selected artifact is `ds-inspect.gif`. First identify the demonstrated mechanism: body rotation, hinge change and pose locking. Next identify the LCD source: the current edition's replayed native opening. Finally identify the destination: animated repository page or first-frame book still.

A defensible caption is: “DS body and hinge inspection with a session pose lock; recorded opening on the LCDs.” In the book, append the first-frame limitation. Do not write “the game responds while the player folds the DS.” That sentence would require separate live native-input evidence.

ND12-R01 — Retry after the condition changes

Now assume the GIF is valid but a controls panel covers the lamp during the brightness change. Decoding it again cannot repair the composition. In an authorized future capture, dismiss or reposition the panel through the real UI, retake the affected sequence, then repeat temporal review and integrity checks. Do not paint over the panel or substitute a brighter render.

If publication alone fails, preserve the distinction: verify the exact source and delivery gate rather than recording replacement footage to fix an unrelated hosted assertion. The present chapter performs neither retry; it specifies the decision rule.

ND12-Q01 — Does the book still prove that the light followed the lamp?

Explained answer

ND12-A01

Back to question

No. One frame can show a lamp and an illuminated region together. Motion requires the original sequence or separate temporal evidence. The source also explains that the light is attached to the movable root, but source structure is not a substitute for human review of the animation.

Source trail and edition boundary

The relevant public records are 115: immediate loading, 116: upright lighting, 117: earlier fresh media and 120: replacement opening edition. They document intent and historical receipts, not newly reproduced results. The four GIF links across these two chapters reuse current DS/shared scene assets unchanged. No historical dirty PNG, new photograph, regenerated artwork or new game capture is part of this manuscript.

PART 4 / EVIDENCE AND MAINTENANCE

ND13

The maintainer field guide

> *Locate the boundary. Follow the mechanism. Inspect the evidence.*

Contents

The maintainer field guide

Question: where should a maintainer start when the DS is wrong? **Outcome:** select a bounded investigation, preserve evidence and change the component that owns the failure instead of hiding the symptom.

Start with the visible contract

Record which cartridge or original fixture, which view, which input path and which source revision produced the failure. State whether the result was local, hosted CI or the actual deployed website. Do not start by editing a timeout or downgrading resolution.

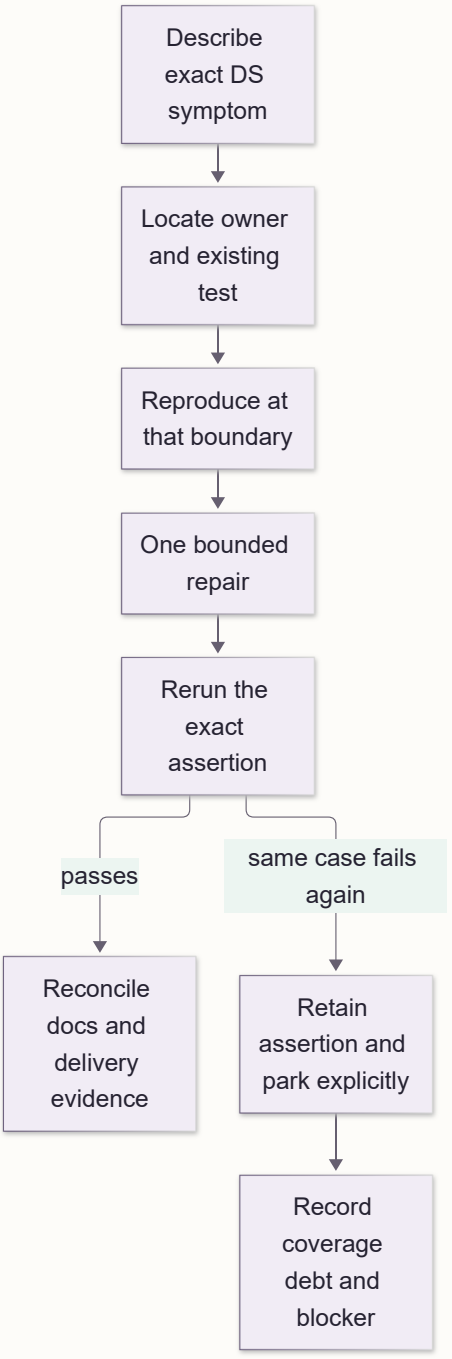


Figure ND13-01. The bounded recovery policy prevents an endless repair loop and prevents green dashboards from silently erasing an unresolved assertion.

A compact fault map

Symptom	First owner to inspect	Evidence that matters
Upload remains in tools	`DsController.load` ordering	Playing view and advancing loading LCD before a held ROM read resolves

First startup fails	Adapter loader and launch callbacks	Visible CDN/error message, cancellation and retry
One screen duplicates the other	`dsScreenRegions`, `paintScreens`	Native dimensions and distinct actual upper/lower output
Quick tap disappears	`DsTouchQueue`, native frame polling	Native contact at the clicked pixel, not only a DOM event
Touch drifts after rotation	`touchAt`, camera/model transforms	Actual lower-LCD ray and native mark under the pointer
Direction releases too early	Adapter source sets	Mixed keyboard/pointer ownership and final release
Progress does not persist	`DsSaveStore`, `persist`	Completed transaction and matching runtime/ROM identity
Upright shell disappears	Scene lamp target and playing pose	Shell pixels under the actual lamp, with readable LCDs retained
A stylus stays held after leaving	Cleanup and availability guards	Release/cancel, hidden held clone and returned docked model

These are starting points, not a promise that every symptom has one cause. Inspect the actual source and existing assertions before changing ownership.

Run the smallest relevant command

The application requires Node 22 or newer and its existing locked dependencies. From the repository root:

```
npm run build
npm run check
node --test tests\unit\ds-cartridge.test.mjs tests\unit\ds-controls.test.mjs tests\unit\ds-touch.test.mjs
```

Build creates the allowlisted static preview. Check performs the repository's source checks. The selected unit files cover DS format/control/touch logic, not native game compatibility. Restore dependencies only when missing or after a dependency-manifest change.

The managed browser preview serves built `dist`; rebuild after source changes. Use a unique existing test name instead of a Windows file path interpreted as a regular expression. For example:

```
npx playwright test --grep "DS upload leaves the sidebar"
```

This is a reproduction recipe, not a claim that this book-authoring task reran application tests. Preserve the existing viewport, deadlines, workers/retries and quality settings unless a separately justified change requires them. A named skipped case is not newly passing because a grep selects it.

ND13-W01 - Change touch-edge rounding

First inspect `dsTouchFromUv` and its unit cases. Predict UV zero, UV one, mid-screen and nonfinite/out-of-range behavior. Then inspect adapter `dispatchTouch`: its combined-canvas pixel-center rule is a second transformation, not a duplicate of the first.

Run focused unit cases, then the native pointer scenarios needed by the change: quick tap, held pen, changed orientation, zoom and high-DPI touch. Record any parked cases explicitly. A center-point screenshot alone cannot qualify an edge-rounding change.

Keep current guides current; keep editions historical

The player guide answers how to operate the current console. Architecture describes current boundaries. Verification records identified observations. The development ledger explains decisions and failed approaches. Numbered workstreams describe a finite outcome, its evidence and actual remaining work. This book explains the cutoff implementation in a coherent narrative.

When behavior changes, update the current guide and affected explanation, then create a clearly identified new rendered edition when necessary. Do not silently overwrite the original GBA book or re-label this edition's historical source pin. Keep the root README short and visual, linking readers to the detailed documentation.

ND13-R01 - A documentation-only correction

If only a caption or prose claim changes, inspect its source and rerun the documentation/book gates affected by the edit. Do not recapture every GIF or run unrelated native suites merely to make the change look rigorous. If the book builder changes, its own tests and actual rendered output are relevant; parser success alone is not layout review.

ND13-Q01 - When is the workstream complete?

Explained answer

ND13-A01 - When the promised outcome has its own evidence

Back to question

Implementation, targeted validation, documentation and owned continuity must agree. Where publication is authorized, verify the exact remote revision and artifact rather than assuming a push deployed it. Mirror the actual workstream in its issue and Project item. Keep human

acceptance, remaining compatibility limits and parked tests explicit instead of using one DONE label to imply all of them.

Source trail

Declared application commands. DS browser assertions. Touch queue unit evidence. Bounded parked-case record.

Source register and edition limits

What is pinned

Implementation and historical citations point to public repository revision `61845372142b41992916f3732ea102cc0f4c8edf`. This is the runtime cutoff, not the later commit that publishes this book. The production source manifest hashes the actual manuscript, selected illustrations and builder inputs. It also records the pinned source files referenced by chapters.

The book synthesizes source-backed behavior and a selected causal history. It is not a complete export of every workstream, issue, transcript or development record. No private conversation or context store is a source. Unfinished historical states remain unfinished in the original records; later public qualification is described separately.

Implementation map

Chapters	Source and symbols	What the source supports
03, 04, 07, 10	ds-controller.js: `load`, `launch`, `message`, `command`, `save`, `suspend`, `togglePower`	DS lifecycle, session correlation, early loading handoff, power semantics and explicit errors
03, 05, 06, 07	player.js: `initialize`, `capture`, `restore`, `input`, `dsInputFrame`, `dispatchTouch`	Pinned native adapter, retained framebuffer, source-owned input and native touch scheduling
04-07, 09	ds-cartridge.js: `DS_OPTIONS`, `identifyDsRom`, `dsScreenRegions`, `dsTouchFromUv`, backup encoder/decoder	Accepted cartridge/header ranges, display geometry, coordinate conversion and portable envelope
06, 09, 10	ds-touch.js: `DsTouchQueue`	Contact-edge preservation, adjacent-move coalescing, cancellation generation and error delivery
07, 09, 10	ds-store.js: `DsSaveStore`	Separate database, runtime/ROM identity, bounded open and transaction-complete acknowledgement
06	preferences.js: `CONTROLS`, with DS overrides	Actual DS key mapping, not legacy canned status wording
02, 08	ds-handheld.js	Physical lid/base, control anchors, markings, slots and stylus geometry

08	scene.js, desk-clearance.js	Playing/inspection pose, actual lamp targeting and mode-specific spatial clearance

These are explanations of the inspected application at the cutoff. They do not independently certify DeSmuME internals or future runtime versions.

Historical and evidence map

Chapters	Public record	Role and limits
01-02	79 - DS implementation, subsequent DS workstreams and ledger	Causal development evidence, including locally verified stages and open owner/device review
01, 11	114 - supported-console status	Later product designation; not universal title or device qualification
02, 04, 08	115 - immediate loading, 116 - upright lighting	Specific user-facing problems, bounded changes and dated local evidence
11, 13	105 - parked browser cases	Explicit unresolved coverage, not repaired cases
11	Verification, exact linked workflow runs	Identified local/hosted/public observations, not a fresh rerun during book authoring
06, 11, 13	DS browser tests, touch unit tests	Original Twin Lab correctness contracts and explicit skip boundaries
12	120 - untouched opening media, Media, capture/replay scripts	Current showcase provenance; recorded LCD replay is not live input evidence

Included illustrations

The cover reuses supplied `assets/ds-logo.png` without regeneration. Four chapter figures select existing `ds-inspect.gif`, `ds-lighting.gif`, `ds-gameplay.gif` and `axolotl-movement.gif`. Print and offline HTML embed metadata-clean frame-zero PNG derivatives. This makes the reader independent of animation playback and the network; it also means a still cannot establish motion, a complete lighting comparison or an actual gameplay response.

The animated source remains available in the repository. Game output is historical showcase footage with recorded native LCD replay. No ROM, raw video or private filesystem path is embedded. The project notices remain the artwork/dependency provenance reference. This edition does not assert new ownership of supplied logos, game imagery or Nintendo marks.

Reproducibility and review boundary

The repository's locked local book dependencies render Mermaid and paginate with Chromium and Paged.js. PyMuPDF checks the real PDF. Installed system fonts and browser versions can affect pagination and bytes; a fresh-machine or byte-identical rebuild is not claimed.

Native text coverage, page geometry, link destinations, bookmarks and embedded fonts are mechanical checks. Contact sheets and selected larger native pages are a separate bounded visual review. Neither is user approval, PDF/UA conformance, printer prepress, a rights audit or new device testing. The production/review record names the exact delivered edition and limits.